RQF LEVEL 3



GENCP302

NETWORKING AND INTERNET TECHNOLOGIES

C Programming Fundamentals

TRAINER'S MANUAL

October, 2024

C PROGRAMMING FUNDAMENTALS

KOICA Korea International Cooperation Agency

TQUM TVET Quality Management Project

2024

AUTHOR'S NOTE PAGE (COPYRIGHT)

The competent development body of this manual is Rwanda TVET Board ©, reproduce with permission.

All rights reserved.

- This work has been produced initially with the Rwanda TVET Board with the support from KOIKA through TQUM Project
- This work has copyright, but permission is given to all the Administrative and Academic Staff of the RTB and TVET Schools to make copies by photocopying or other duplicating processes for use at their own workplaces.
- This permission does not extend to making of copies for use outside the immediate environment for which they are made, nor making copies for hire or resale to third parties.
- The views expressed in this version of the work do not necessarily represent the views of RTB. The competent body does not give warranty nor accept any liability
- RTB owns the copyright to the trainee and trainer's manuals. Training providers may reproduce these training manuals in part or in full for training purposes only. Acknowledgment of RTB copyright must be included on any reproductions. Any other use of the manuals must be referred to the RTB.

© Rwanda TVET Board

Copies available from:

- *HQs: Rwanda TVET Board-RTB*
- *Web: www.rtb.gov.rw*
- **KIGALI-RWANDA**

Original published version: October2024

ACKNOWLEDGEMENTS

The publisher would like to thank the following for their assistance in the elaboration of this training manual:

Rwanda TVET Board (RTB) extends its appreciation to all parties who contributed to the development of the trainer's and trainee's manuals for the TVET Certificate III in Networking and Internet Technologies, specifically for the module **"GENCP302: C Programming Fundamentals."**

We extend our gratitude to KOICA Rwanda for its contribution to the development of these training manuals and for its ongoing support of the TVET system in Rwanda

We extend our gratitude to the TQUM Project for its financial and technical support in the development of these training manuals.

We would also like to acknowledge the valuable contributions of all TVET trainers and industry practitioners in the development of this training manual.

The management of Rwanda TVET Board extends its appreciation to both its staff and the staff of the TQUM Project for their efforts in coordinating these activities.

This training manual was developed:

Under Rwanda TVET Board (RTB) guiding policies and directives



Under Financial and Technical support of



COORDINATION TEAM

RWAMASIRABO Aimable

MARIA Bernadette M. Ramos

MUTIJIMA Asher Emmanuel

PRODUCTION TEAM

Authoring and Review

UWAMAHORO Bonaventure

KWIZERA Jean Paul

Validation

MVUYEKURE Jean Damascene

MANISHIMWE Yves

Conception, Adaptation and Editorial works

HATEGEKIMANA Olivier

GANZA Jean Francois Regis

HARELIMANA Wilson

NZABIRINDA Aimable

DUKUZIMANA Therese

NIYONKURU Sylvestre

NYINAWUMUNTU Gaudence

Formatting, Graphics, Illustrations, and infographics

YEONWOO Choe

SUA Lim

SAEM Lee

SOYEON Kim

WONYEONG Jeong

KAYUMBA Fiston

Financial and Technical support

KOICA through TQUM Project

TABLE OF CONTENT

AUTHOR'S NOTE PAGE (COPYRIGHT)---iii
ACKNOWLEDGEMENTS---iv
TABLE OF CONTENT ---vii
ACRONYMS---viii
INTRODUCTION ---1
MODULE CODE AND TITLE: GENCP302 C PROGRAMMING FUNDAMENTALS---2
Learning Outcome 1: Describe Computer Programming Language ---3
Key Competencies for Learning Outcome 1: Describe Computer Programming Languages 4
Indicative content 1.1: Identification of Programming Language 6
Indicative content 1.2: Development of a Flowchart 9
Indicative content 1.3: Development of an Algorithm 12
Learning outcome 1 end assessment.....16
Further information to the trainer.....25
Learning Outcome 2: Write C Programming Codes ---26
Key Competencies for Learning Outcome 2: Write C Programming Codes.....27
Indicative content 2.1: Description of C Building Blocks 29
Indicative content 2.2: Description of C Program Structure.....36
Indicative content 2.3: Applications of Condition Statements 45
Indicative content 2.4: Applications of Loops.....53
Indicative content 2.5: Applications of Functions.....60
Indicative content 2.6: Applications of Arrays 64
Indicative content 2.7: Solving of C Programming Problems 69
Learning outcome 2 end assessment.....75
Further information to the trainer.....93
Learning Outcome 3: Perform Program Debugging---94
Key Competencies for Learning Outcome 3: Perform Program Debugging 95
Indicative content 3.1: Identification of Errors.....97
Indicative content 3.2: Compilation of the C Program 102
Indicative content 3.3: Test of the C Program 106
Learning outcome 3 end assessment.....109
Further information to the trainer.....118

ACRONYMS

ALGOL: Algorithmic Language

ALU: Arithmetic Logic Unit

API: Application Programming Interface

APL: A Programming Language

ASCII: American Standard Code for Information Interchange

BASIC: Beginners All Purpose Symbolic Instruction Code

COBOL: Common Business Oriented Language

CSS: Cascading Style Sheets

DLL: Dynamic Link Library

DMA: Direct Memory Access

EOF: End of File

FORTRAN: Formula Translation

GCC: GNU Compiler Collection

GPU: Graphics Processing Unit

GUI: Graphical User Interface

IDE: Integrated Development Environment

IO: Input/output

IOT: Internet of things

JIT: Just-In-Time compilation

JSON: JavaScript Object Notation

MVC: Model-View-Controller

OOP: Object-Oriented Programming

OS: Operating System

PL/I: Programming Language, Version 1

Prolong: Programing Logic

PSU: Power Supply Unit

RPG: Report Program Generator

INTRODUCTION

This trainer's manual includes all the methodologies required to effectively deliver the module titled **"C Programming Fundamentals."** Trainees enrolled in this module will engage in practical activities designed to develop and enhance their competencies.

The development of this training manual followed the Competency-Based Training and Assessment (CBT/A) approach, offering ample practical opportunities that mirror real-life situations.

The trainer's manual is organized into Learning Outcomes, which is broken down into indicative content that includes both theoretical and practical activities. It provides detailed information on the key competencies required for each learning outcome, along with the objectives to be achieved.

As a trainer, you will begin by asking questions related to the activities to encourage critical thinking and guide trainees toward real-world applications in the labor market. The manual also outlines essential information such as learning hours, didactic materials, and suggested methodologies.

This manual outlines the procedures and methodologies for guiding trainees through various activities as detailed in their respective trainee manuals. The activities included in this training manual are designed to offer students opportunities for both individual and group work. Upon completing all activities, you will assist trainees in conducting a formative assessment known as the end learning outcome assessment. Ensure that students review the key reading and the points to remember section.

MODULE CODE AND TITLE: GENCP302 C PROGRAMMING FUNDAMENTALS

Learning Outcome 1: Describe Computer Programming Language
Learning Outcome 2: Write C Programming Codes
Learning Outcome 3: Perform Program Debugging


C++
C
Ruby
Perl
C++
Java
Pascal
Delphi
Program
C++
Python
Python

Indicative contents
1.1 Identification of programming Language
1.2 Development of a flowchart
1.3 Development of an algorithm

Key Competencies for Learning Outcome 1: Describe Computer Programming Languages

Knowledge	Skills	Attitudes
• Description of programming Languages. • Identification of programming IDE. • Description of flowchart. • Description of algorithm.	• Installing programming IDE • Analysing a given problem • Arranging steps (logical sequence) according to the given problem • Designing flowchart • Developing an algorithm	• Being Patient • Have Critical thinking, and Attentive • Having Team Work and creativity • Having Team Work, and spirit

Duration: 20 hrs

Learning outcome 1 objectives:

By the end of the learning outcome, the trainees will be able to:

1. Describe correctly different types of programming languages as used in computer programming
2. Install properly C programming IDE according to the computer specifications.
3. Describe correctly different flowchart symbols according to their functions.
4. Design properly the flowchart according to the problem to be solved.
5. Develop properly the algorithms according to the problem to be solved.

Resources

Equipment	Tools	Materials
● Computers ● Projector ● Projection screen ● Printers	● Internet access ● C programming IDE	● Chalks ● Pens ● Books ● Papers ● Flip-chart

Advance Preparation:

Before delivering this learning outcome, you are recommended to:

- Avail setup of C programming IDE.
- To have a demonstration video show the installation processes of C programming IDE.
- Avail IDE with integrated C compiler installation wizard as a guidance.

Indicative content 1.1: Identification of Programming Language

Duration: 5 hrs

Theoretical Activity 1.1.1: Description of programming languages

Notes to the trainer:

- While delivering this activity, small groups may be used to identify different types of computer programming.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and ask trainees to do the following questions:

i. What is a computer programming language?
ii. What is a computer programming IDE?
iii. Enumerate 2 role of programming IDE in telecommunications
iv. Give 2 types of programming languages.

Step 2: Monitor the discussion and asks trainees to write the findings on paper.

Step 3: Ask trainees to present their findings to the whole class.

Step 4: Provide an expert view and relate it to what they presented.

Step 5: Address any questions or concerns.

Step 6: Ask them to read the Key readings **1.1.1 in their manuals**.

Points to Remember

Types of programming Languages

- A computer programming language is a formal language used to write instructions that a computer can execute.
- There are two types: **low-level languages**, which are closely tied to hardware, and **high-level languages**, which offer more abstraction.

- Programming languages are essential in telecommunications for developing firmware and software for embedded systems, network monitoring, and automation.
- C Programming is a widely-used, easy-to-learn general-purpose language that helps understand core programming concepts, making it easier to learn other high-level languages.
- Integrated Development Environments (IDEs) are tools where programmers write code; C has various IDEs compatible with different operating systems like Windows, macOS, and Linux, with some being cross-platform.



Practical Activity 1.1.2: Installation of C programming IDE



Notes to the trainer

- Facilitation of this activity may be individual based; you are supposed to demonstrate how to Install C programming IDE.
- it is recommended to avail C IDE setup and computers.



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to read the task described below:

John has a computer which doesn't have C Programming IDE and he needs to write a program which will display" TVET SCHOOL". You are asked to install C Programming IDE in that computer.

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Demonstrate how to install C Programming IDE while demonstrating, explain all steps to install C Programming IDE.

Step 4: Ask trainees to reinstall C Programming IDE and monitor the activity.

Step 5: Verify whether the program has been installed

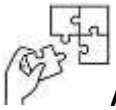
Step 6: Ask trainees to read key reading **1.1.2.**

Step 7: Ask trainees to perform the task provided in application of learning 1.1

Points to Remember

Setup the environment if C programming IDE

- Visit the official website of the IDE (e.g., Visual Studio, Code::Blocks, Eclipse) and download the appropriate installer or Locate the setup of C programming IDE .
- Install a C compiler (e.g., GCC for Linux/Mac, MinGW for Windows) if the IDE doesn't include one.
- Follow the installation instructions provided by the installer, adjusting settings as needed.
- Set up the IDE to use the correct compiler and debugger.
- Configure paths for libraries and include files if necessary.
- Create a simple C program to ensure everything is working correctly.
- Compile and run the program to get the output.



Application of learning 1.1.

Madam Rose needs to write a C program to display her name, but her computer lacks a C programming IDE. you are requested install a C programming IDE on her computer.

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1	IDE is properly installed	**1.1.**IDE is selected to the media		
		1.2. The steps of installation are processed		
		1.3. The installation is tested		

Indicative content 1.2: Development of a Flowchart

Duration: 5hrs

Theoretical Activity 1.2.1: Description of the flowchart

Notes to the trainer:

- While delivering this activity, small groups may be used for description of the flowchart symbols and steps for developing flowchart.
- Avail different photo to demonstrate how different flowchart symbols are looking like.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and ask trainees to do the following question:

i. What is a flowchart?
ii. What are the importance of a flowchart in visualizing processes.
iii. What are symbols used to design a flowchart.

Step 2: Monitor the discussion and asks trainees to write the findings on paper
Step 3: Ask trainees to present their findings to the whole class.
Step 4: Provide an expert view and relate it to what they presented.
Step 5: Address any questions or concerns.
Step 6: Ask them to read the Key readings **1.2.1 in their manuals.**

Points to Remember

- Flowcharts are diagrammatic representations of processes.
- Flowchart Symbols ovals, rectangles, diamonds, and arrows to show the sequence of steps, decision points, and outcomes. They simplify complex processes, enhance communication, and aid in problem-solving, documentation, and training.



Practical Activity 1.2.2: Designing a flowchart



Notes to the trainer

- While delivering this activity, individual task and/or small group may be used.
- The trainer may demonstrate how design flowchart
- Trainer may use trainer guided to demonstrate the steps followed for designing flowchart



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to read the task described below:
Using a flowchart Describe the steps involved in going to the cinema to watch a movie, including actions like choosing a film, buying tickets, and making decisions, such as verifying ShowTime availability.
Step 2: Demonstrate how to design flowchart with suitable example.
Step 3: Asks learners to be active in their groups and monitor the procedures.
Step 4: Ask trainees to present their findings to the whole class.
Step 5: Provide an expert view and relate it to what they presented.
Step 6: Ask trainees to read key **reading 1.2.2.**
Step 7: Ask trainees to perform the task provided in application of learning 1.2



Points to Remember

Design a flowchart

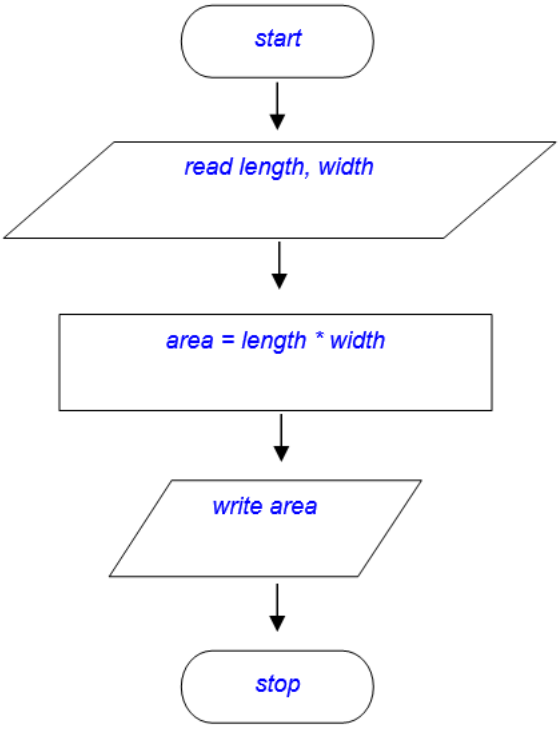
- Define the Objective
- Use Standard Symbols
- Maintain Logical Flow

Application of learning 1.2.

In southern province different sectors want to construct the playground for each school. They need calculate the area of each ground using its length and width. Draw a flowchart that will accept the length, width to calculate and print the area of the playground. (area = length * width).

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1	The appropriate symbols are correctly selected	**1.1.**Rectangle is selected		
		1.2. Oval is selected		
		1.3. All symbols are in proper place		
2	The flowchart is correctly designed	**2.1.**The flowchart is to the given question.		



Indicative content 1.3: Development of an Algorithm

Duration: 10 hrs

Theoretical Activity 1.3.1: Description of algorithms types.

Notes to the trainer:

- While delivering this activity, the trainer may use small groups.
- Avail examples of written algorithms to describe types of algorithms.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and request learners to answer to the following questions:

i. What is an algorithm?
ii. What is pseudo code, and how is it used in the algorithm design process?
iii. Enumerates the different types of algorithms.

Step 2: Asks any learner to write their findings on flipchart/whiteboard/papers.

Step 3: Asks learners to discuss the provided answer and choose correct answers.

Step 4: Provides expert view and clarifies ideas by using didactic materials.

Step 5: Address any questions or concerns.

Step 6: Ask trainees to read the key **reading 1.3.1** in the trainee manual.

Points to Remember

An algorithm is a sequence of instructions designed to solve a problem or achieve a specific task. It can be translated into a programming language to produce a desired result.

Structure of an Algorithm:

- Variable Declaration: Defines variables used in the algorithm.
- Beginning: Marks the start of the algorithm.
- Instructions: Contains the sequence of steps to be executed.
- End: Marks the completion of the algorithm.
    - **Types of Algorithms:** Sequence, Branching (Selection), Loop (Repetition):
- **Pseudo Code:**
    - ✓ An informal, text-based language for designing algorithms.
    - ✓ Helps programmers plan and outline solutions before coding.
    - ✓ Rules are straightforward and aid in algorithmic design.



Practical Activity 1.3.2: Development of an algorithm



Notes to the trainer

- The trainer may use individual based; you are supposed to demonstrate how to develop an algorithm.
- Avail practical examples of developing an algorithm



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to read the task described below:

> You are tasked to develop an algorithm and pseudocode to sort books on a library bookshelf by their titles alphabetically. The library currently has a disorganized shelf where books are placed randomly, and the librarian wants them arranged in alphabetical order for easier access and browsing by library patrons.

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)
Step 3: Demonstrate steps used todevelop an algorithmand explain each step
Step 4: Asktrainees to develop an algorithm and monitor the activity.
Step 5: Test a developed algorithm.
Step 6: Ask trainees to read key reading **1.3.2.**and to perform application of learning **1.3**



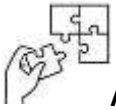
Points to Remember

- **Develop an algorithm**

✓ Obtain detailed information on the problem.
✓ Analyse the problem.
✓ Think of a problem-solving approach.
✓ Review the problem-solving approach and try to think of a better Alternative.
✓ Develop a basic structure of the Algorithm.
✓ Optimize, Improve and refine.

- **Write the pseudo code**

✓ Always capitalise the initial word (often one of the main six constructs).
✓ Make only one statement per line.
✓ Indent to show hierarchy, improve readability, and show nested constructs.
✓ Always end multi-line sections using any of the END keywords (ENDIF, ENDWHILE, etc.).
✓ Keep your statements programming language independent?
✓ Use the naming domain of the problem, not that of the implementation. For instance: “Append the last name to the first name” instead of “name = first+ last.”
✓ Keep it simple, concise and readable.



Application of learning 1.3.

The GB company is located in Kigali, they pay their employees using paper. Due to this issue the employees did get easily their payroll statement. As a programmer Write an algorithm and pseudocode to calculate and display the total monthly expenses based on various categories like rent, groceries, utilities, and entertainment. The user should input expenses for each category.

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1	**Algorithm is well written**	**1.1.**The algorithm steps are written		
		1.2. The written algorithm is as answer of agiven question		
2	**Pseudocode iscorrectly written**	**2.1.**The pseudocode is written		
		2.2. The pseudocode answers the given questions		

Algorithm:

1. Declare variables for each category and total expenses.
2. Start the process.
3. Prompt the user to enter the expense for each category.
4. Read the expenses and store them in respective variables.
5. Calculate the total expenses.
6. Display the total expenses.
7. End the process.

Pseudocode:

Declare Rent, Groceries, Utilities, Entertainment, Total as Float

Start

Write("Enter rent expense:")
Read(Rent)
Write("Enter groceries expense:")
Read(Groceries)
Write("Enter utilities expense:")
Read(Utilities)
Write("Enter entertainment expense:")
Read(Entertainment)
Total = Rent + Groceries + Utilities + Entertainment
Write("Total monthly expenses: ", Total)
End



Learning outcome 1 end assessment

Written assessment

1. What is a symbol in a flowchart represents the start and end of the process?
   Answer: The Oval symbol.
2. What is a programming language?
   Answer: A programming language is a formal language comprising a set of instructions used to produce various outputs and implement algorithms.
3. Explain the purpose of a flowchart in software development.
   Answer: A flowchart is used to visually represent the logic and sequence of an algorithm or process, making it easier to understand and communicate the steps involved in solving a problem or performing a task.
4. Describe the steps you would follow to install a C programming IDE like Code::Blocks on your computer.
   Answer: Steps include downloading the installer, running it, following the installation wizard, selecting the installation directory and components, installing the software, launching the IDE, configuring the compiler if necessary, and creating a new project to verify the installation.
5. Compare and contrast the use of procedural programming and object-oriented programming paradigms in software development.
   Answer: Procedural programming focuses on a sequence of instructions and is ideal for tasks with a clear procedure, while object-oriented programming organizes code into objects that encapsulate data and behavior, making it suitable for large-scale applications requiring modularity and reuse.
6. Evaluate the pros and cons of using an interpreted language like Python for developing a web application.
   Answer: Pros include easier debugging, platform independence, and rapid prototyping. Cons include slower execution compared to compiled languages and potential runtime errors that may not be detected until execution.
7. Name two programming languages commonly used for embedded systems development.
   Answer: C and C++.
8. What is the primary function of pseudo code in the development process?
   Answer: Pseudo code is used to describe the logic and flow of a program in an informal way, bridging the gap between an algorithm and its implementation in a programming language.

9. Given a scenario where you need to automate a repetitive task in a telecommunications system, which programming language would be most appropriate and why?
   Answer: A language like Python could be appropriate due to its simplicity, extensive libraries, and support for automation and scripting.
10. Analyze the use of flowcharts versus pseudo code in a team setting for a large-scale project. Which would you prefer and why?
    Answer: Flowcharts might be preferred for their visual clarity and ease of understanding for all team members, while pseudo code may be more detailed and closer to the final implementation, aiding in smoother transitions to coding.
11. Copy and complete the following table

Name of the symbol	**Diagram**	**Description**
Oval	……	……
……	……	It is used for data processing
……		……
Circle	……	……
……		……

Answer:

Name of the symbol	Diagram	Description
Oval	Oval	Denotes the start and end of the flowchart
Rectangle		It is used for data processing
Rhombus or Diamond		Denotes a decision (branch)to be made.The program should continue along one of two routines.True of False
Circle		Can be used to eliminate lengthy of flowrines. It's used to indicate that one symbol is connected to another.
Parallelogram		Denotes an Input operations

12. Compare and contrast different programming paradigms (procedural, object-oriented, functional, and logic programming). Provide examples of languages that exemplify each paradigm and discuss their primary use cases.

Answer:

- **Procedural Programming:** Focuses on a sequence of instructions to perform tasks. It uses functions, loops, and conditionals. Example languages include C and Pascal. Primary use cases are system programming, embedded systems, and procedural tasks.
- **Object-Oriented Programming (OOP):** Organizes code into objects, which are instances of classes. OOP emphasizes encapsulation, inheritance, and polymorphism.

Example languages include Java, C++, and Python. Primary use cases are large-scale software systems, GUI applications, and simulations.

- **Functional Programming:** Treats computation as the evaluation of mathematical functions, avoiding changing state and mutable data. Example languages include Haskell, Lisp, and Scala. Primary use cases are concurrent and parallel processing, data transformation tasks, and scenarios requiring high reliability.
- **Logic Programming:** Uses formal logic to express computation. Programs consist of a set of facts and rules that describe relationships and logic to infer conclusions. Example languages include Prolog and Datalog. Primary use cases are artificial intelligence, natural language processing, and problem-solving applications that involve complex rule-based logic.

13. Describe the differences between compiled and interpreted programming languages. Discuss the advantages and disadvantages of each approach with examples of languages that use these methods.

Answer: Compiled and interpreted languages differ in how they execute code.

Compiled Languages:

- **Process:** Source code is translated into machine code by a compiler before execution.
- **Execution:** The machine code is directly executed by the hardware.
- **Performance:** Generally faster execution due to direct machine code execution.
- **Error Detection:** Errors are detected during the compilation process.
- **Distribution:** Only the compiled machine code needs to be distributed, not the source code.
- **Examples:** C, C++, Rust.

Advantages:

- High performance and efficiency.
- Improved security as the source code is not exposed.
- Error detection at compile-time helps catch issues early.

Disadvantages:

- Longer development cycle due to the compilation step.
- Platform-specific, requiring different binaries for different systems.

Interpreted Languages:

- **Process:** Source code is translated into machine code line-by-line by an interpreter at runtime.
- **Execution:** The interpreter executes the translated code directly.
- **Performance:** Slower execution due to the overhead of translation at runtime.
- **Error Detection:** Errors are detected at runtime, potentially making debugging more challenging.
- **Distribution:** Source code or an intermediate representation needs to be distributed.
- **Examples:** Python, JavaScript, Ruby.

Advantages:

- Easier debugging and testing with immediate feedback.
- Platform-independent as the interpreter can run on any system.
- More flexibility and ease of development, suitable for scripting and rapid prototyping.

Disadvantages:

- Slower execution compared to compiled languages.
- Runtime errors can occur, which might not be caught until the code is executed.

Multiple choice questions

14. Which of the following is an advantage of compiled languages over interpreted languages?
a) Easier to debug
b) Faster execution
c) Platform independence
d) Immediate feedback
Answer: b) Faster execution

15. Which of the following steps would be the best starting point for creating a new software application?
a) Writing the final code
b) Designing a flowchart or pseudo code
c) Debugging the code
d) Compiling the program
Answer: b) Designing a flowchart or pseudo code

16. In what scenario would functional programming be preferred over logic programming?
a) Concurrent processing tasks
b) Artificial intelligence applications
c) Designing embedded systems
d) Web development
Answer: a) Concurrent processing tasks

17. Which IDE would you choose for C/C++ development if you needed a lightweight, customizable environment?
a) Eclipse
b) Visual Studio
c) Code::Blocks
d) CLion
Answer: c) Code::Blocks

18. Pseudo code is primarily used for:
a) Debugging code
b) Writing final programs
c) Describing the steps in an algorithm
d) Designing hardware
Answer: c) Describing the steps in an algorithm

Practical assessment

1.You need to write an algorithm and pseudocode for creating a grocery shopping list. The program should allow the user to input items they need to buy, then display the complete list of items.

- **Algorithm:**

1. Declare an empty list to store items.
2. Start the process.
3. Prompt the user to enter an item.
4. Read the item and add it to the list.
5. Ask if the user wants to add another item.
6. If yes, repeat steps 3-5.
7. If no, display the complete list of items.
8. End the process.

- **Pseudocode**:

  Declare an empty list to store items.

Start the process.
Prompt the user to enter an item.
Read the item and add it to the list.
Ask if the user wants to add another item.
If yes, repeat steps 3-5.
If no, display the complete list of items.
End the process.

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1.	The algorithm is correctly written	**1.1.** The algorithm is aligned in logical sequence		
		1.2. The logical sequence of the algorithm solves the given situation		
2.	Pseudocode is correctly written	**2.1.** The Pseudocodes are aligned in logical sequence		
		2.2. the logical sequence of the pseudocode solves the given problem		

2. Develop an algorithm and pseudocode for a library management system to add and display books. The user can enter the book's title and author, and the system should store and display the list of books.

- **Algorithm:**

1. Declare a list to store books.
2. Start the process.
3. Prompt the user to enter the title and author of a book.
4. Read the title and author, and store them in a list as a book.
5. Ask if the user wants to add another book.
6. If yes, repeat steps 3-5.
7. If no, display the list of books.
8. End the process.

- **Pseudocode:**

```
Declare Books as List
Start
Do
  Write("Enter book title:")
  Read(Title)
  Write("Enter book author:")
  Read(Author)
  Add (Title, Author) to Books
```

```
    Write("Do you want to add another book? (yes/no)")
    Read(Response)
  While Response = "yes"
  Write("List of books in the library:")
  For Each Book in Books
    Write("Title: ", Book.Title, " Author: ", Book.Author)
  End
```

Checklist/solution for application

SN	CRITERIA	INDICATORS	YES	NO
1.	The algorithm is correctly written	**1.1.** The algorithm is aligned in logical sequence		
		1.2. The logical sequence of the algorithm solves correctly the given situation		
2.	Pseudocode is correctly written	**2.1.** The Pseudocodes are aligned in logical sequence		
		2.2. the logical sequence of the pseudocode solves the given problem		

3.Create an algorithm and pseudocode for a simple calculator that performs addition, subtraction, multiplication, and division based on user inputs.

- **Algorithm:**

1. Declare variables for the two numbers and the result.
2. Start the process.
3. Prompt the user to enter the first number.
4. Read the first number.
5. Prompt the user to enter the second number.
6. Read the second number.
7. Prompt the user to choose an operation (addition, subtraction, multiplication, division).
8. Perform the chosen operation and store the result.
9. Display the result.
10. End the process.

- **Pseudocode:**

```
Declare Num1, Num2, Result as Float
Declare Operation as String
Start
```

```
Write("Enter the first number:")
Read(Num1)
Write("Enter the second number:")
Read(Num2)
Write("Choose an operation (+, -, *, /):")
Read(Operation)
If Operation = "+"
   Result = Num1 + Num2
Else If Operation = "-"
   Result = Num1 - Num2
Else If Operation = "*"
   Result = Num1 * Num2
Else If Operation = "/"
   If Num2 != 0
      Result = Num1 / Num2
   Else
      Write("Error: Division by zero")
      End
Write("The result is: ", Result)
End
```

Checklist/solution for application

SN	CRITERIA	INDICATORS	YES	NO
1.	The algorithm is well written	**1.1.** The algorithm is aligned in logical sequence		
		1.2. The logical sequence of the algorithm solves correctly the given situation		
2.	Pseudocode is well writen	**2.1.** The Pseudocodes are aligned in logical sequence		
		2.2. the logical sequence of the pseudocode solves the given problem		

END



Further information to the trainer

A. Books

Graham, D. L. (2019). C Programming Language: A Step by Step Beginner's Guide to Learn C Programming. Amazon.

King, K. (2008). C Programming: A Modern Approach 2nd Edition. Amazon.

McGrath, M. (2018). C Programming in easy steps, 5th Edition. Amazon.

Miller, G. P. (2013). C Programming Absolute Beginner's Guide. Amazon.

Norton, W. (2021). The C Programming Language (Simplified for Beginners). Amazon.

Thareja, R. (2011). Data Structures Using C. Amazon.

B. Websites

https://www.scribd.com/presentation/413537916/Lect1-Algorithms-and-Flowchart-ppt?utm_medium=cpc&utm_source=google_pmax&utm_campaign=3Q_Google_Performance-Max_RoW&utm_term=&utm_device=c&gclid=CjwKCAjw36GjBhAkEiwAKwIWyYBOAwSwzUDmki-xCos35zYEx-s7uQqw4Q2rHSpf2FG2VySd7fYxMRoCmJAQAvD_BwE#

https://faradars.org/wp-content/uploads/2015/07/algorithm-and-flow-chart.pdf

https://terrorgum.com/tfox/books/flowchartandalgorithmbasics.pdf

https://www.w3resource.com/c-programming-exercises/conditional-statement/index.php


evaluated
statement
standard
case
void
contains
array
constant
variables
function
POINTER
size
word order
zero
define
int
read
char
USED
CHARACTER
INITIALIZED
PROGRAM
include
TYPE
CALLED
return
operator
double
structure
time
write
String
specifier
SPACE
STRUCT
input
error
library
object
expression
RESULT
declaration
external
NULL

Indicative contents
2.1. Description of C programming building blocks
2.2. Description of C program structure
2.3. Application of condition statements
2.4. Application of loops
2.5. Application of functions
2.6. Application of Arrays
2.7. Solving a problem using

Key Competencies for Learning Outcome 2: Write C Programming Codes

Knowledge	Skills	Attitudes
• Description of C structure • Description of control structure • Differentiating interpreter from compiler • Differentiate statement from expression • Differentiation variable from constants • Differentiation different data types • Description of functions • Identification of the scope of a variable	• Installing C based IDE • Applying variables • Using Operators • Using different data types. • Applying conditional statements. • Applying functions • Applying control structure • Applying Loops • Applying Arrays • Document program • Using comments	• Having Critical thinking • Being Passionate • Having Team work spirit • Have Persistence • Being Hard working • Being Self-motivated • Having Self confidence



Duration: 50 hrs



Learning outcome 2 objectives:

By the end of the learning outcome, the trainees will be able to:

1. Describe correctly the building blocks as per C programming language
2. Describe correctly the C program structure as per functional program
3. Apply properly a condition statements based on a syntax and certain conditions
4. Apply properly C loops based on syntax and certain instructions
5. Apply properly C functions based on syntax and specified task
6. Apply properly C Arrays based on syntax, data store and their use.
7. Solve a real-world problem in line with C programming language



Resources

Equipment	Tools	Materials
● Computer ● Projector	● Visual Studio Code ● Dev-C++ ● Python tutor code visualizer ● Internet	● Electricity



Advance Preparation:

Before delivering this learning outcome, you are recommended to:

- Prepare sample Illustrations, images and Videos.
- Organize classroom /or computer lab
- Avail Computer with C based IDE

Indicative content 2.1: Description of C Building Blocks

Duration: 5hrs

Theoretical Activity 2.1.1: Description of C Tokens

Notes to the trainer:

- Trainer may small groups may be used to describe C Tokens.
- Avail different photo to demonstrate how different C token are looking like.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and ask trainees to do the following questions:

i. What is a token in the context of the C programming language?
ii. How are tokens used to represent fundamental building blocks in C code?
iii. Name and describe the five main categories of C tokens.
iv. What are identifiers in C, and how do they differ from keywords?
v. Give examples of valid and invalid identifiers in C.
vi. Explain the role of keywords in C and provide some examples.
vii. Name the three primary types of constant tokens in C.

Step 2: Monitor the discussion and asks trainees to write the findings on paper

Step 3: Ask trainees to present their findings to the whole class.

Step 4: Provide an expert view and relate it to what they presented.

Step 5: Address any questions or concerns.

Step 6: Ask them to read the Key readings **2.1.1**

Points to Remember

In C programming, tokens are fundamental units that the compiler interprets. They encompass keywords, identifiers, constants, operators, punctuation symbols, whitespace, and comments, serving as the basic building blocks of C syntax:

- **Keywords:** Reserved words with predefined meanings, like int, float, if, else, etc.
- **Identifiers:** User-defined names for variables, functions, etc., following specific rules and case-sensitive.
- **Constants:** Fixed values like integer (123), floating-point (3.14), character ('A'), and string literals ("Hello").
- **Operators:** Symbols performing operations (arithmetic, relational, logical, bitwise, assignment, etc.) on variables.
- **Punctuation Symbols:** Special characters (, ,, ;, : :, (), {} {}, []) that structure C code.
- **Whitespace:** Spaces, tabs, newlines used for token separation, ignored by the compiler.
- **Comments:** Provide non-executable information to programmers (// Single-line, /* Multi-line */).



Practical Activity 2.1.2: Applying the Data types in a c



Notes to the trainer

- The trainer may use individual based; you are supposed to demonstrate how to write the Data types in a c program.
- Avail visual aids, such as images, text, or code examples, to illustrate the concept of C variable declaration.



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to read the task described below:

John, a computer science student, is tasked with developing a program that manages inventory for a small retail store. His program needs to handle different types of data, from item quantities to prices, and ensure accurate calculations and storage of information. Write all data types John needs to complete his task.

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Ask trainees to discuss by listing all the needed data types.

Step 4: Ask trainees to write down the findings

Step 5: Ask trainees to Present the findings to the class.

Step 6: Ask trainees to read key reading **2.1.2 Trainee Manual**

 **Points to Remember**

Data type in c programming

- **Choosing Appropriate Data Types**: Selecting the right data type (int, float, char, etc.) based on the nature and range of the data ensures proper storage and manipulation.
- **Variable Declaration and Initialization**: Variables must be declared with a specific data type, and initializing them at the time of declaration ensures they start with a defined value.
- **Operations and Expressions**: Data types dictate how operations (arithmetic, relational, logical, bitwise) are performed and how values are interpreted.
- **Type Casting**: Type casting allows conversion between data types, with implicit casting done automatically and explicit casting requiring programmer intervention.
- **Memory Allocation**: Data types determine the amount of memory allocated to variables, affecting how arrays and structs manage contiguous memory.

 **Theoretical Activity 2.1.3: Description of Variables in C**

 **Notes to the trainer:**

- Trainer may use small group to describe c variables
- Apply variables in c programming

 **Key steps:**

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to answer the task described below:

i. Name the three fundamental variables in C.

ii. what is the Rules to write variable names?

iii. Explain the differences between local, global and static variables in C.

iv. What is the syntax for declaring and initialising variables in C?

v. Explain the importance of initialising variables when declaring them.

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Ask trainees to discuss on given tasks by writing the findings

Step 4: Ask trainees to present to the whole class

Step 5: Ask trainees to read key reading **2.1.3**



Points to Remember

Variables in c programming

- **Variable Naming Rules**: Variable names must start with an alphabet, can include numbers, but cannot contain special characters or spaces (except the underscore) and should not be a reserved word.
- **Declaration of Variables**: Variables must be declared before use, specifying the data type (e.g., int x;).
- **Assigning Values**: Values can be assigned to variables either during or after declaration using the assignment operator (=).
- **Types of Variables**: Variables in C can be local, global, or static, each with different scopes and lifetimes.
- **Local vs. Global Variables**: Local variables are accessible only within the block they're declared in, while global variables are accessible throughout the entire program.



Theoretical Activity 2.1.4: Description of constants



Notes to the trainer:

- Trainer may small groups to describe the constants
- Avail the examples of constants used in programming.



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to answer the task described below:

i. What is a constant in C programming?

ii. Why are constants used in programming, and what benefits do they offer?

iii. Name and describe the three main types of constants in C.

iv. Explain the format and syntax of integer constants in C.

v. Describe how floating-point constants are represented in C.

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Demonstrate the use of constraint in a c program

Step 4: Aske trainees to discuss on the given topic and write down their finding.

Step 5: Ask trainees to present their finding to the whole class

Step 6: Provide more clarification of the presentation

Step 7: Ask trainees to read key reading **2.1.4 Trainee Manual**



Points to Remember

Constants in C Programming

Constants are fixed values that do not change during the execution of a program, and they are also known as literals. C programming supports several types of constants namely:

- ✓ Integer Constants
- ✓ Character Constants
- ✓ String Constants



Theoretical Activity 2.1.5: Description of operators



Notes to the trainer:

- Trainer may small groups to describe the operators
- Avail the examples of operators used in programming.



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to answer the following questions:

i. What are operators in programming?

ii. Describe the types of operators in programming

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Demonstrate the use of operators in a c program

Step 4: Aske trainees to discuss on the given topic and write down their finding.

Step 5: Ask trainees to present their finding to the whole class

Step 6: Provide more clarification of the presentation

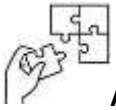
Step 7: Ask trainees to read key reading **2.1.5 Trainee Manual**



Points to Remember

C programming operators

- ✓ **Arithmetic Operators** (+, -, *, /, %) for basic math operations.
- ✓ **Relational Operators** (==, !=, >, <, >=, <=) for comparing values.
- ✓ **Logical Operators** (&&, ||, !) for combining or negating conditions.
- ✓ **Bitwise Operators** (&, |, ^, ~, <<, >>) for manipulating data at the bit level.
- ✓ **Assignment Operators** (=, +=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=) for assigning and updating values.
- ✓ **Unary Operators** (+, -, ++, --, !, ~) for operations on a single operand.
- ✓ **Ternary Operator** (?:) for conditional decisions.
- ✓ **Special Operators** (sizeof, &, *, ->, .) for size checking, memory access, and structure manipulation.



Application of learning 2.1.

Ann is a programmer with career, she is going to hire another to help him, the following is the written exam to pass before to be hired.Declare and initialize the following variables with appropriate data types and values.

1. An integer variable year with a value of 2024.
2. A float variable pi with a value of 3.14.
3. A double variable gravity with a value of 9.81.
4. A character variable grade with a value of 'A'.
5. A string variable greeting with the value "Hello, World!"

Solution:

```
#include <stdio.h>

int main() {
   int year = 2024;
```

```
    float pi = 3.14f;
    double gravity = 9.81;
    char grade = 'A';
    char greeting[] = "Hello, World!";
printf("Year: %d\n", year);
printf("Pi: %.2f\n", pi);
printf("Gravity: %.2lf\n", gravity);
printf("Grade: %c\n", grade);
printf("Greeting: %s\n", greeting);

    return 0;
}
```

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1.	The variables are correctly declared	**1.1.** The integer is declared		
		1.2. The float is declared		
		1.3. Double are declared		
		1.4. Character are declared		
		1.5.string are declared		

Indicative content 2.2: Description of C Program Structure

Duration: 5 hrs

Theoretical Activity 2.2.1: Description of Pre-processor Commands

Notes to the trainer:

- Trainer small groups may be used to identify different types of computer programming.
- Avail different photo to demonstrate how different computers programming are looking like.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and ask trainees to do the following questions:

i. What is the purpose of pre-processor directives in C?

ii. How is the #include directive used to include header files in C code?

iii. What is the difference between including header files with angle brackets (<>) and double quotes (" ")?

iv. How do you define a constant using the #define directive, and what are the advantages of doing so?

v. What is conditional compilation, and how is it achieved using pre-processor directives in C?

Step 2: Monitor the discussion and asks trainees to write the findings on papers.

Step 3: Ask trainees to present their findings to the whole class.

Step 4: Provide an expert view and relate it to what they presented.

Step 5: Address any questions or concerns.



Points to Remember

Preprocessor commands in c

Preprocessor commands, also known as preprocessor directives, are used in programming languages like C and C++ to control the behavior of the preprocessor, which is a component of the compiler. Common preprocessor commands include:

- **#include**: Used to include the contents of a header file into the source code, allowing access to external code or libraries (e.g., #include <stdio.h>).
- **#define**: Used to create macros, which are symbolic names for values or code fragments that are replaced by their definitions during preprocessing (e.g., #define PI 3.14159265).
- **#ifdef, #ifndef, #else, #endif**: Used for conditional compilation, allowing code to be included or excluded based on preprocessor-defined conditions (e.g., #ifdef DEBUG).
- **#undef**: Used to undefine a previously defined macro (e.g., #undef DEBUG_MODE).
- **#error**: Generates a compilation error with a custom message, useful for enforcing certain conditions during compilation (e.g., #error "MY_CONST is not defined!").
- **#pragma**: Provides additional information to the compiler or controls certain compiler-specific behavior. Its usage can vary between different compilers.
- **#line**: Changes the line number and filename reported by the compiler, often used in code generation tools and for debugging.



Theoretical Activity 2.2.2: Description of Functions



Notes to the trainer:

- Trainer may use small groups to describe functions.
- Avail different images to demonstrate how different computer programming functions are looking like.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and ask trainees to do the following questions:

i. What is a function in C, and why is it used in programming?
ii. How do you declare a function in C?
iii. Explain the difference between types of function in C.
iv. Give an example of a types function.
v. What is the purpose of a return statement in a C function, and how is it used?

Step 2: Monitor the discussion and asks trainees to write the findings on papers.

Step 3: Ask trainees to present their findings to the whole class.

Step 4: Provide an expert view and relate it to what they presented.

Step 5: Address any questions or concerns.

Step 6: Ask them to read the Key readings **2.2.2**

Points to Remember

Functions in C

Definition of a Function

A function in C is a block of code that performs a specific task, helping to organize, improve readability, and reuse code.

Types of Functions in C

1. **Library Functions**: Predefined in C standard libraries (e.g., printf(), scanf()).
2. **User-defined Functions**: Created by the programmer to perform custom tasks.

Theoretical Activity 2.2.3: Description of Variables

Notes to the trainer:

- Trainer may use small groups to describe types of variables.
- Avail examples of written algorithms to describe types of variables.

 **Key steps:**

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and request learners to respond to the following questions:

- **i.** What is a variable in the context of C programming, and why are they important?
- **ii.** How do you think variables help us store and manipulate data in a C program? Can you provide examples?
- **iii.** Why is it important to choose meaningful names for variables? What could go wrong if you use vague or unclear names?
- **iv.** What are some of the basic data types in C, and how do they affect the types of values a variable can store?
- **v.** When you declare a variable, what information must you specify, and why is it necessary?
- **vi.** Differentiate declaring and defining a variable in C? What's the key distinction?
- **vii.** What happens if you try to use a variable before declaring it? Why is it important to declare a variable before using it?
- **viii.** How do you assign a value to a variable in C programming? Can you provide an example?
- **ix.** Why do you think it's essential to understand the scope of a variable in a program? How can variable scope impact your code?
- **x.** Describe the process of initialising a variable in C, and why is it considered best practices?

Step 2: Monitor the discussion and asks trainees to write the findings on paper

Step 3: Ask trainees to present their findings to the whole class.

Step 4: Provide an expert view and relate it to what they presented.

Step 5: Address any questions or concerns.

Step 6: Trainer orient them to read the key reading **2.2.3 Trainee Manual**

 **Points to Remember**

Variables in C Programming

Variables are fundamental components in C used to store and manipulate data. They represent named memory locations for storing values and come in various data types.

- **Data Types**

C supports several basic data types:

- ✓ **int**: Stores integer values.
- ✓ **char**: Stores characters, such as letters and symbols.
- ✓ **float**: Stores single-precision floating-point numbers.
- ✓ **double**: Stores double-precision floating-point numbers.
- ✓ **short**: Stores short integers.
- ✓ **long**: Stores long integers.
- ✓ **unsigned**: Stores non-negative integer values.
- ✓ **void**: Used for functions that return no value.



Practical Activity 2.2.4: Apply Variables in C programming



Notes to the trainer

- The trainer may use small groups to apply variables in c programming.
- Avail the examples of variable application in c.



Key steps:

While delivering this activity, pass through the following steps:

Step 1:	Introduce the topic and ask learners to read tasks described below: In your formed group apply the concepts of variables, data types, initialization, and arithmetic operations in C programming by creating a simple program to calculate the areas of a rectangle and a circle.
Step 2:	Explain the task and provide clear work instruction (Task, Time allocated)
Step 3:	Ask trainees to discuss how initialize, and the use of data type in a given task.
Step 4:	Ask trainees to write their finding on paper
Step 5:	Ask trainees to present their findings to the whole class
Step 6:	Ask trainees to read key **reading 2.2.4 Trainee Manual.**



Points to Remember

Applying variables in C

- **Variable Declarations**: Variables in C are declared by specifying a data type followed by the variable name (e.g., int age;, char grade;).
- **Multiple Declarations**: Multiple variables of the same type can be declared on a single line (e.g., int x, y, z;).
- **Initialization**: Variables can be declared and initialized in a single line (e.g., int count = 10;, char initial = 'A';).
- **Constants**: Use the const keyword to declare constants whose values cannot be changed (e.g., const int numberOfDaysInAWeek = 7;).
- **Global Variables**: Variables declared outside any function are global and accessible throughout the program (e.g., int globalVar = 42;).
- **Local Variables**: Variables declared within a function are local and accessible only within that function (e.g., int localVar = 100;).
- **Dynamic Memory Allocation**: Dynamically allocate memory using pointers and malloc (e.g., int *dynamicArray = (int *)malloc(10 * sizeof(int));).



Theoretical Activity 2.2.5: Description of Statements & Expressions



Notes to the trainer:

- Trainer may use small group to describe statements and expressions.
- Avail examples of expression usage.



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and ask trainees to do the following questions:

i. What is the fundamental difference between a statement and an expression in C programming, and can you provide examples of each?

ii. How are statements used to control the flow of execution in a C program? Can you give an example of a control statement and its purpose?

iii. Explain the role of expressions in C programming. How are expressions evaluated, and what are some common examples of expressions?

Step 2: Monitor the discussion and asks trainees to write the findings on papers.
Step 3: Ask trainees to present their findings to the whole class.
Step 4: Provide an expert view and relate it to what they presented.
Step 5: Address any questions or concerns.
Step 6: Ask them to read the Key readings **2.2.5 Trainee Manual**

 **Points to Remember**

Statements & Expressions

- **Statements:** A statement is a complete line of code that performs an action, typically ending with a semicolon and does not return a value but may have side effects.
- **Expressions:** An expression combines variables, constants, operators, and function calls to produce a single value and always returns a value but does not cause side effects.
- **Statement Examples:** Includes assignment (e.g., x = 10;), control flow (e.g., if (condition) {...}), and function calls (e.g., printf("Hello, World");).
- **Expression Examples:** Includes arithmetic (e.g., x + y), logical (e.g., a && b), and function call expressions (e.g., sqrt(25)).
- **Relationship:** Statements often contain expressions to calculate values or make decisions, such as in assignment or control flow statements.

 **Theoretical Activity 2.2.6: Description of Comments**

 **Notes to the trainer:**

- Trainer may small group describe comments.

 **Key steps:**

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and request learners to respond to the following questions:

i. What are comments in C programming, and what is their primary purpose when writing code?

ii. Explain the different types of comments used in C programming, such as single-line comments and multi-line comments. Can you provide examples of each?

Step 2: Monitor the discussion and asks trainees to write the findings on paper

Step 3: Ask trainees to present their findings to the whole class.

Step 4: Provide an expert view and relate it to what they presented.

Step 5: Address any questions or concerns.

Step 6: Trainer orients them to read the key reading **2.2.6 Trainee Manual**



Points to Remember

Comments in c programming

- **Types of Comments:** In C, comments can be single-line (using //) or multi-line (enclosed in /* */).
- **Code Explanation:** Use comments to clarify complex or non-intuitive code, making it easier to understand.
- **Documentation:** Comments can describe the purpose, usage, and behavior of functions, variables, and data structures.
- **Code Organization:** Comments can help separate and organize different sections of code for better readability.
- **Best Practices:** Comments should be clear, concise, up-to-date, used sparingly, and focus on explaining the "why" rather than the "what" of the code.



Application of learning 2.2.

Rwanda BGD want to manage your daily expenses. As you are a programmer, declare two integer variables representing amounts spent on two different items. Perform addition, subtraction, multiplication, and division on these amounts to understand your total and average spending. Print the results to see your calculations.

Solution:

```
#include <stdio.h>

int main() {
   // Declare two integer variables for daily expenses
   int expense1 = 50;  // Amount spent on groceries
   int expense2 = 30;  // Amount spent on transport
```

```
   // Perform arithmetic operations
   int total = expense1 + expense2;
   int difference = expense1 - expense2;
   int product = expense1 * expense2;
   float quotient = (float)expense1 / expense2;

   // Print the results of arithmetic operations
printf("Total expenses: %d\n", total);
printf("Difference in expenses: %d\n", difference);
printf("Product of expenses: %d\n", product);
printf("Quotient of expenses: %.2f\n", quotient);
   return 0;
}
```

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1.	Variable are correctly declared	**1.1.** Expense1 is declared		
		1.2. Expense1 is declared		
2.	Operator are correctly used	**2.1.** Addition operator is used		
		2.2. Multiplication operator is used		

Indicative content 2.3: Applications of Condition Statements

Duration: 15 hrs

Theoretical Activity 2.3.1: Description of the conditional statement in C

Notes to the trainer:

- Trainer may use small group to describe the conditional statement in c programming
- Avail examples of written algorithms to describe the conditional statement in c programming.
- Link the conditional statement with the daily life examples.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and request learners to respond to the following questions:

i. What is a conditional statement, and why is it an essential concept in programming?

ii. Provide an example of a real-life situation where you would use a conditional statement to make a decision or perform different actions based on a specific condition.

iii. Explain the 'if' statement in C programming, including how it works, its syntax, and its purpose? Additionally, clarify the concept of 'else if' statements and their use cases compared to multiple 'if' statements, and describe when and how to use nested 'if' statements in your code."

iv. What is a "switch" statement, and how does it differ from "if" statements?

v. Describe the conditions or scenarios in which you would choose to use "if" statements over "switch" statements, and vice versa.

Step 2: Monitor the discussion and asks trainees to write the findings on paper

Step 3: Ask trainees to present their findings to the whole class.

Step 4: Provide an expert view and relate it to what they presented.

Step 5: Address any questions or concerns.

Step 6: Trainer orients them to read the key reading **2.3.1 Trainee Manual**

Points to Remember

Conditional statement in c programming

Essential for decision-making in code, allowing different actions based on specific conditions.

1. **'if' Statement in C:**
    - Used to execute code blocks only if a condition is true. The syntax includes if (condition) { code }.
    - else if statements allow checking multiple conditions, providing more control than multiple standalone if statements.
    - Nested if statements enable more complex decision-making by placing if statements inside other if statements.
2. **Switch Statements:**
    - Used for handling multiple possible values of a single variable, often simpler and more readable than using multiple if statements.



Practical Activity 2.3.2: Solving problem using if statement in c



Notes to the trainer

- The trainer may use small groups to solve problem using if statement.
- Avail daily life examples where if statement is used.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to read tasks described below:
You need to implement a feature in a shopping application that checks if a customer is eligible for a discount based on the amount they spend. The rules for the discount are as follows:
    - If the total purchase amount is greater than or equal to $100, the customer receives a 10% discount.
    - If the total purchase amount is less than $100, no discount is applied.

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Ask trainees to discuss by writing the pseudocode of a given task.

Step 4: Ask trainees to convert the written pseudocode into program code by using if statement syntax.

Step 5: Ask trainees to present their finding to the class.

Step 6: Ask trainees to ask clarification where necessary.

Step 7: **Step 6:** Ask trainees to read key **reading 2.3. Trainee Manual**



Points to Remember

If statement in C programming

An if statement in C programming is a control structure used to execute a block of code only if a specified condition is true.

Syntax of if Statement

```
if (condition) {
   // code to execute if condition is true
}
```

Components

- **Condition**
- **Block of Code**



Practical Activity 2.3.3: Solving problem using else if statement in c programming



Notes to the trainer

- The trainer may use small groups to solve problem using if statement.
- Avail daily life examples where else if statement is used.



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to read tasks described below:
You need to implement a feature in an educational application that assigns a letter grade to a student based on their score. The grading criteria are as follows:

- A score of 90 or above receives an "A".
- A score of 80 to 89 receives a "B".
- A score of 70 to 79 receives a "C".
- A score of 60 to 69 receives a "D".
- A score below 60 receives an "F".

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Ask trainees to discuss by writing the pseudocode of a given task.

Step 4: Ask trainees to convert the written pseudocode into program code by using else if statement syntax.

Step 5: Ask trainees to present their finding to the class.

Step 6: Ask trainees to ask clarification where necessary.

Step 7: Ask trainees to read key **reading 2.3. Trainee Manual**



Points to Remember

else if statement in c

The else if statement in C programming is used to handle multiple conditional branches in a decision-making process

Syntax of else if

```
if (condition1) {
   // Code to execute if condition1 is true
} else if (condition2) {
   // Code to execute if condition1 is false and condition2 is true
} else if (condition3) {
   // Code to execute if condition1 and condition2 are false and condition3 is true
} else {
   // Code to execute if none of the above conditions are true
}
```

Components

- **Initial if Statement**
- **else if Statements**
- **Optional else Statement**



Practical Activity 2.3.4: Solving problem using switch statement in c programming



Notes to the trainer

- The trainer may use small groups to solve problem using if statement.
- Avail daily life examples where else if statement is used.



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to perform tasks described below:
You want to create a simple calculator application that allows users to choose an arithmetic operation (addition, subtraction, multiplication, or division) from a menu. The program should then perform the selected operation on two numbers provided by the user.

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Ask trainees to discuss by writing the pseudocode of a given task.

Step 4: Ask trainees to convert the written pseudocode into program code by using switch statement syntax.

Step 5: Ask trainees to present their finding to the class.

Step 6: Ask trainees to ask clarification where necessary.

Step 7: Ask trainees to read key **reading 2.3. 4 Trainee Manual**



Points to Remember

Switch Statement

Executes one code block among many options based on a variable's value, offering a cleaner alternative to multiple if...else if statements.

- **Expression Evaluation:**

The switch statement evaluates an expression that must be an integer or character, and this value is compared against case labels.

- **Case Labels:**

Case labels represent possible values for the expression. If a match is found, the associated code block is executed.

- **Break Statement:**

  Used to exit the switch block after a case is executed; prevents "fall-through" to subsequent cases unless intended.

- **Default Case:**

  Provides a fallback option if no case matches; optional but recommended to handle unexpected values.



Practical Activity 2.3.5: Solving problem using nested conditions in c



Notes to the trainer

- The trainer may use small groups to solve problem using if statement.
- Avail daily life examples where else if statement is used.



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to read tasks described below:
You need to decide what to wear based on the weather conditions and the type of event you plan to attend. The choices depend on whether it is a formal event or a casual outing, combined with the weather being sunny, rainy, or cold.

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Ask trainees to discuss by writing the pseudocode of a given task.

Step 4: Ask trainees to convert the written pseudocode into program code by using nested if syntax.

Step 5: Ask trainees to present their finding to the class.

Step 6: Ask trainees to ask clarification where necessary.

Step 7: Ask trainees to read key **reading 2.3.5 Trainee Manual**

Points to Remember

Nested conditions in C refer to using conditional statements, such as if, else if, and else, within other conditional statements.

```
if (condition1) {
   // Code block executed if condition1 is true
   if (condition2) {
      // Code block executed if condition1 and condition2 are true
   } else {
      // Code block executed if condition1 is true and condition2 is false
   }
} else {
   // Code block executed if condition1 is false
   if (condition3) {
      // Code block executed if condition1 is false and condition3 is true
   } else {
      // Code block executed if condition1 and condition3 are false
   }}
```



Application of learning 2.3.

ABC Company seller of movie. They need to hire a programmer to Write a program that calculates the price of a movie ticket based on the customer's age and the time of day they wish to watch the movie. The program should apply different pricing rules based on these conditions.

Solution:

```
#include <stdio.h>

int main() {

   int age, time;

   double ticketPrice;

   // Prompt the user to enter their age

printf("Enter your age: ");

scanf("%d", &age);
```

```
    // Prompt the user to enter the movie time
printf("Enter the movie time (24-hour format): ");
scanf("%d", &time);
    // Determine the base ticket price based on age
    if (age < 13) {
ticketPrice = 5.0; // Child price
    } else if (age < 18) {
ticketPrice = 7.0; // Teenager price
    } else {
ticketPrice = 10.0; // Adult price
    }
    // Apply discount based on the time
    if (time < 17) { // Matinee discount
ticketPrice *= 0.8; // Apply 20% discount
    }
    // Print the final ticket price
printf("Final ticket price: $%.2f\n", ticketPrice);


    return 0;
}
```

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1.	Nested if is correctly applied	**1.1.** The condition one is identified		
		1.2. The condition two is identified		

Indicative content 2.4: Applications of Loops

Duration: 10 hrs

Theoretical Activity 2.4.1: Description of Loops

Notes to the trainer:

- Trainer may use small groups to describe types of loops.
- Avail examples of written algorithms to describe types of loops.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and request learners to respond to the following questions:

i. What is a loop in programming, and why do we use loops?
ii. How does a "while" loop work, and what's its primary purpose?
iii. What are the key differences between a "for" loop and a "while" loop?
iv. Can you provide a practical example of when you might use a loop in a real-life situation?
v. Why is it important to have a clear exit condition for loops to prevent them from running infinitely?
vi. How does a loop help in avoiding repetitive and redundant code in a program?

Step 2: Monitor the discussion and asks trainees to write the findings on paper

Step 3: Ask trainees to present their findings to the whole class.

Step 4: Provide an expert view and relate it to what they presented.

Step 5: Address any questions or concerns.

Step 6: Trainer orients them to read the key reading **2.4.1 Trainee Manual**

Points to Remember

Loops in c programming

- **For Loop:**

Used when the number of iterations is known beforehand; includes initialization, condition, and increment/decrement in one line.

- **While Loop:**

  Ideal for scenarios where the number of iterations is not predetermined; continues as long as the condition is true.

- **Do-While Loop:**

  Similar to a while loop but ensures the loop body executes at least once before the condition is checked.

- **Loop Control Statements:**

  break exits the loop immediately, continue skips the current iteration, and goto transfers control to a labeled statement.



Practical Activity 2.4.2: Solving problem using for loop in c programming



Notes to the trainer

- The trainer may use small groups to solve problem using for loop.
- Avail daily life examples where for loop is used.



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to perform tasks described below:
You are developing a simple voting system where users can vote for different candidates. After collecting the votes, you need to count and display the number of votes each candidate received.

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Ask trainees to discuss by writing the pseudocode of a given task.

Step 4: Ask trainees to convert the written pseudocode into program code by using for loop syntax.

Step 5: Ask trainees to present their finding to the class.

Step 6: Ask trainees to ask clarification where necessary.

Step 7: Ask trainees to read key **reading 2.4.2.Trainee Manual**

Points to Remember

Structure of a for Loop in C

In C programming, the for loop has the following general structure:

```
for (initialization; condition; increment)
{
   // Code to be executed on each iteration
}
```



Practical Activity 2.4.3: Solving problem using while loop in c programming



Notes to the trainer

- The trainer may use small groups to solve problem using while loop.
- Avail daily life examples where else while loop is used.



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to perform tasks described below:
You are developing a basic ATM system that allows users to withdraw money from their account. The system needs to repeatedly prompt the user to enter the amount they wish to withdraw, ensure the amount does not exceed their balance, and check if it is a valid withdrawal amount. The loop should continue until the user successfully withdraws the amount or decides to cancel the transaction.

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Ask trainees to discuss by writing the pseudocode of a given task.

Step 4: Ask trainees to convert the written pseudocode into program code by using while syntax.

Step 5: Ask trainees to present their finding to the class.

Step 6: Ask trainees to ask clarification where necessary.

Step 7: Ask trainees to read key **reading 2.4.3 Trainee Manual**

Points to Remember

Structure of a while Loop in C

In C programming, the while loop has the following general structure:

```
while (condition) {
   // Code to be executed on each iteration
}
```

Practical Activity 2.4.4: Solving problem using do while in c programming

Notes to the trainer

- The trainer may use small groups to solve problem using do while.
- Avail daily life examples where do while is used.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to perform tasks described below:
You are developing a user authentication system that prompts the user to enter their password. The system should continue to prompt the user until they enter the correct password or choose to cancel after a certain number of failed attempts. You want to ensure that the prompt is displayed at least once, regardless of whether the password is correct on the first try.

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Ask trainees to discuss by writing the pseudocode of a given task.

Step 4: Ask trainees to convert the written pseudocode into program code by using do while syntax.

Step 5: Ask trainees to present their finding to the class.

Step 6: Ask trainees to ask clarification where necessary.

Step 7: Ask trainees to read key **reading 2.4.4 Trainee Manual**

Points to Remember

Structure of a do-while Loop in C

The general structure of a do-while loop in C is:

```
do {
   // Code to be executed on each iteration
} while (condition);
```

Practical Activity 2.4.5: Solving problem using nested loops in c

Notes to the trainer

- The trainer may use small groups to solve problem using nested loops.
- Avail daily life examples where nested loops is used.

Key steps:

While delivering this activity, pass through the following steps:

Step 1:	Introduce the topic and ask learners to perform tasks described below: You are developing a program that needs to generate and display a multiplication table for numbers from 1 to 10. This involves creating a table where each cell in the table represents the product of the row and column numbers.
Step 2:	Explain the task and provide clear work instruction (Task, Time allocated)
Step 3:	Ask trainees to discuss by writing the pseudocode of a given task.
Step 4:	Ask trainees to convert the written pseudocode into program code by using nested loop syntax.
Step 5:	Ask trainees to present their finding to the class.
Step 6:	Ask trainees to ask clarification where necessary.
Step 7:	Ask trainees to read key **reading 2.4.5. Trainee Manual**

Points to Remember

Structure of Nested Loops

In most programming languages, including C, nested loops are structured as follows:

```
for (int i = 0; i< n; i++) {          // Outer loop
   for (int j = 0; j < m; j++) {       // Inner loop
      // Code to be executed
   }
}
```

Application of learning 2.4

You are organizing a 3-day conference in ATL hotel in Kigali with multiple sessions each day. Each session has multiple activities, such as presentations, workshops, and discussions. You need to create a schedule that covers all sessions and their activities over the 3 days. To do this, you'll need to use nested loops to iterate through each day, each session of the day, and each activity in the session.Write a c program make these activities.

Solution:

```
#include <stdio.h>
int main() {
   // Define the days, sessions, and activities
   char *days[3] = {"Day 1", "Day 2", "Day 3"};
   char *sessions[3] = {"Morning Session", "Afternoon Session", "Evening Session"};
   char *activities[3] = {"Presentation", "Workshop", "Discussion"};
   // Iterate through each day
   for (int i = 0; i< 3; i++) {
printf("%s:\n", days[i]); // Print the day


      // Iterate through each session of the day
```

```
        for (int j = 0; j < 3; j++) {
printf("  %s:\n", sessions[j]); // Print the session

            // Iterate through each activity in the session
            for (int k = 0; k < 3; k++) {
printf("    - %s\n", activities[k]); // Print the activity
            }
        }
printf("\n"); // Print a newline for better readability between days
    }

    return 0;
}
```

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1.	While loop is correctly used	**1.1.** **Outer Loop** is written		
		1.2. Inner loop is sated		

Indicative content 2.5: Applications of Functions

Duration: 5 hrs

Theoretical Activity 2.5.1: Description of types of C functions

Notes to the trainer:

- Trainer may use small groups to demonstrate c functions
- Avail examples of c functions.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and request learners to respond to the following questions:

i. What is a function in programming, and why do we use them?
ii. Can you give an example of a situation in everyday life that's similar to how a function works in C?
iii. What are some common functions you've encountered or used in C programming?
iv. How do you call a function in C, and what does it mean to "call" a function?
v. What is the purpose of a return statement in a function, and how is it used?

Step 2: Monitor the discussion and asks trainees to write the findings on paper

Step 3: Ask trainees to present their findings to the whole class.

Step 4: Provide an expert view and relate it to what they presented.

Step 5: Address any questions or concerns.

Step 6: Trainer orients them to read the key reading **2.5.1 Trainee Manual**

Points to Remember

Function in c

A function in C is a block of code designed to perform a specific task, helping organize and manage code effectively.

- **Library Functions:**

Predefined functions provided by C libraries, such as scanf(), printf(), and sqrt().

- **User-Defined Functions:**

  Functions created by the programmer, involving a declaration (prototype), call, and definition, promoting code modularity and reuse.

- **Passing Parameters:**

  Parameters are declared in the function definition, while arguments are the values passed during function calls.

- **Calling Methods:**

  Call by Value passes a copy of the argument's value, while Call by Reference passes the argument's address, affecting the original argument.



Practical Activity 2.5.2: Applying C functions to solve a problem



Notes to the trainer

- The trainer small groups to demonstrate the application of functions in c programming
- Avail examples on function definition, declaring and calling



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and request learners to respond to the following questions:

You are developing a simple student grade management system for a school. The system needs to perform several tasks:

Add Student Grades: Record grades for multiple subjects for each student.

Calculate Average Grades: Compute the average grade for each student.

Determine Grade Category: Categorize students based on their average grade (e.g., Excellent, Good, Average, Needs Improvement).

Step 2: Monitor the discussion and asks trainees to write the findings on paper

Step 3: Ask trainees to present their findings to the whole class.

Step 4: Provide an expert view and relate it to what they presented.

Step 5: Address any questions or concerns.

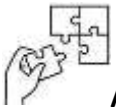
Step 6: Trainer orients them to read the key reading **2.5.2** Trainee Manual



Points to Remember

To apply functions effectively in C:

- **Modular Design:** Break programs into smaller, manageable functions.
- **Parameter Passing:**
  - ✓ Use **pass-by-value** for simple types.
  - ✓ Use **pass-by-reference** (via pointers) for complex types or to modify data.
- **Return Values:** Use return to send a result back. Use void if no value is returned.
- **Error Handling:** Include checks (e.g., for division by zero) within functions.
- **Documentation:** Comment on functions to explain their purpose and parameters, enhancing readability and maintainability



Application of learning 2.5.

LDT Company located in Karongi district, need to make a calculator. Write a C program to create a calculator that illustrates the following tasks:

- Defining functions
- Declaring functions
- Calling functions

Solution:

```
#include <stdio.h>

// Function prototypes
int add(int a, int b);
int subtract(int a, int b);
int multiply(int a, int b);
float divide(int a, int b);

int main() {
   int x = 10, y = 5;
printf("Sum: %d\n", add(x, y));
```

```
printf("Difference: %d\n", subtract(x, y));
printf("Product: %d\n", multiply(x, y));
printf("Quotient: %.2f\n", divide(x, y));
   return 0;
}

// Function definitions
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
float divide(int a, int b) {
   if (b != 0) return (float)a / b;
   else { printf("Error: Division by zero\n"); return 0; }
}
```

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1.	C functions are correctly applied	**1.1.** Function Declaration is mentioned		
		1.2. Function Definition is mentioned		
		1.3. Function Call is mentioned		

Indicative content 2.6: Applications of Arrays

Duration: 5 hrs

Theoretical Activity 2.6.1: Describe the Types of arrays

Notes to the trainer:

- Trainer may use small groups to describe the types of arrays
- Avail the Examples of array types

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and request learners to respond to the following questions:

i. What is an array in programming, and why is it used?
ii. How are elements stored in an array, and what is the significance of the index?
iii. What are the benefits of using arrays in your code?
iv. What are some common use cases where arrays are indispensable?

Step 2: Monitor the discussion and asks trainees to write the findings on paper

Step 3: Ask trainees to present their findings to the whole class.

Step 4: Provide an expert view and relate it to what they presented.

Step 5: Address any questions or concerns.

Step 6: Trainer orients them to read the key reading **2.6.1. Trainee Manual**

Points to Remember

Arrays in c Programming:

In C programming, arrays are data structures that store multiple elements of the same type in contiguous memory. The types of arrays include:

- **One-Dimensional Array:** A single row of elements. Declared with one pair of square brackets.
  - ✓ **Example:** int numbers[5];

- **Two-Dimensional Array:** An array of arrays, forming a matrix. Declared with two pairs of square brackets for rows and columns.
    - ✓ **Example:** int matrix[3][4];
- **Multi-Dimensional Array:** Extends beyond two dimensions with multiple levels of brackets.
    - ✓ **Example:** int data[2][3][4];



Practical Activity 2.6.2: Declaring Arrays



Notes to the trainer

- The trainer may use small groups to declare arrays
- Avail examples of arrays declaration



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to perform tasks described below:
You are a teacher managing a class of students. You need to store the grades of your students in a program so you can easily calculate averages, find the highest and lowest grades, and perform other analyses.

Declare all the array in your program using a suitable syntax

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)
Step 3: Ask trainees to discuss by writing the syntax declaration of arrays.
Step 4: Ask trainees to write the code in a given syntax of arrays
Step 5: Ask trainees to present their finding to the class.
Step 6: Ask trainees to ask clarification where necessary.
Step 7: Ask trainees to read key **reading 2.6.2. Trainee Manual**

Points to Remember

Arrays declaration in c

In C programming, arrays are declared by specifying the data type of the elements and the array size. The methods of declaration vary based on the array type:

- **One-Dimensional Array:**

data_typearray_name[array_size];

- **Two-Dimensional Array:**

data_typearray_name[rows][columns];

Practical Activity 2.6.3: Initializing Arrays

Notes to the trainer

- The trainer may use small groups to initialize arrays
- Avail examples to array initialization

Key steps:

While delivering this activity, pass through the following steps:

Step 1:	Introduce the topic and ask learners to perform tasks described below: You are a teacher managing a class of students. You need to store the grades of your students in a program so you can easily calculate averages, find the highest and lowest grades, and perform other analyses.
Step 2:	Explain the task and provide clear work instruction (Task, Time allocated)
Step 3:	Ask trainees to discuss by writing the code for array initialization.
Step 4:	Ask trainees to write the code in a given syntax of arrays
Step 5:	Ask trainees to present their finding to the class.
Step 6:	Ask trainees to ask clarification where necessary.
Step 7:	Ask trainees to read key reading **2.6.3** Trainee Manual



Points to Remember

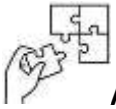
Array initialization in C

Syntax of Array Initialization

The general syntax for initializing an array during its declaration is as follows:

data_typearray_name[array_size] = {value1, value2, value3, ...};

- **data_type:** The type of elements in the array (e.g., int, float, char).
- **array_name:** The name you give to the array.
- **array_size:** The number of elements the array can hold. This can sometimes be omitted if the array is being initialized with values.
- {value1, value2, value3, ...}: A comma-separated list of values used to initialize the array elements.



Application of learning 2.6.

You are a software developer working on a sales analysis application for a retail store in Kigali city. You need to record the sales figures for each day of the week and then calculate the total and average sales. To achieve this, you will declare and initialize an array with the sales data for the week.

Declare and initialize an array to store the daily sales figures for a week (7 days) and then calculate and display the total and average sales.

Solution:

```
#include <stdio.h>

int main() {
   // Declare and initialize an array to store daily sales for a week
   float sales[7] = {1500.75, 2000.50, 1750.25, 1900.60, 2100.30, 1800.90, 2200.10};

   // Calculate the total sales
   float total_sales = 0;
   for(int i = 0; i< 7; i++) {
total_sales += sales[i];
```

```
    }

    // Calculate the average sales
    float average_sales = total_sales / 7;

    // Print the daily sales, total sales, and average sales
printf("Daily Sales:\n");
    for(int i = 0; i< 7; i++) {
printf("Day %d: $%.2f\n", i + 1, sales[i]);
    }
printf("\nTotal Sales: $%.2f\n", total_sales);
printf("Average Sales: $%.2f\n", average_sales);

    return 0;

}
```

Checklist/solution for application

SN	CRITERIA	INDICATORS	YES	NO
1.	Array is correctly initialized	Initialization of array is mentioned		

Indicative content 2.7: Solving of C Programming Problems

Duration: 5 hrs

Practical Activity 2.7.1: Description of draw the flowchart of problem

Notes to the trainer

- The Trainer may small groups to draw the flowchart.
- Avail examples to demonstrate how to draw the flowchart.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to perform tasks described below:
You are a systems analyst working for a bank. Your task is to design a flowchart for the ATM withdrawal process to help developers understand the logic and steps involved in the transaction. Create a flowchart that outlines the steps a user takes to withdraw money from an ATM. The flowchart should include the following steps:

- Start
- Insert ATM card
- Enter PIN
- Validate PIN
- Select withdrawal option
- Enter withdrawal amount
- Check account balance
- Dispense cash if the balance is sufficient
- Display error message if the balance is insufficient
- End transaction and eject card

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)
Step 3: Ask trainees to discuss by writing the findings in appropriate template.
Step 4: Ask trainees to draw a flowchart of a given scenario.
Step 5: Ask trainees to present their finding to the class.
Step 6: Ask trainees to ask clarification where necessary.

Step 7: Ask trainees to read key **reading 2.7.1. Trainee Manual**

Points to Remember

Steps to draw a flowchart

- Understand the Process
- List the Steps
- Choose Flowchart Symbols
- Draw the Flowchart

Practical Activity 2.7.2: Design the algorithm to solve the problem

Notes to the trainer

- The trainer may use small groups to design an algorithm of a problem
- Avail examples of algorithms.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to perform tasks described below:
You are a software developer tasked with creating a system for a school to sort student grades. The system will take a list of student grades and sort them in ascending order to help teachers quickly identify students' performance levels. Design an algorithm that sorts a list of student grades in ascending order

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Ask trainees to discuss by writing the pseudocode of the problem.

Step 4: Ask trainees to design an algorithm of a given scenario.

Step 5: Ask trainees to present their finding to the class.

Step 6: Ask trainees to ask clarification where necessary.

Step 7: Ask trainees to read key **reading 2.7.2 Trainee Manual**

Points to Remember

Steps to design an algorithm

- Understand the Problem
- Plan the Algorithm
- Design the Algorithm



Practical Activity 2.7.3: Write the C program to solve the problem



Notes to the trainer

- The trainer may use small groups to write the c code of a given problem.
- Avail examples of c code.



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to perform tasks described below:
You need to write a C program to find the median of a list of integers. The median is defined as:
- The middle value in a sorted list if the list has an odd number of elements.
- The average of the two middle values in a sorted list if the list has an even number of elements.

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Ask trainees to discuss by writing the pseudocode of the problem.

Step 4: Ask trainees to write the c code of a given scenario.

Step 5: Ask trainees to present their finding to the class.

Step 6: Ask trainees to ask clarification where necessary.

Step 7: Ask trainees to read key **reading 2.7.3 Trainee Manual**



Points to Remember

Writing c code from an algorithm

- Implementing an algorithm in C programming involves using variables, data types, and control structures to execute the steps of the algorithm.
- Well-structured and error-free code is essential for the program to produce the desired results.
- Debugging techniques and error handling are important skills for C programmers to ensure the program functions correctly.



Practical Activity 2.7.4: Program documentation



Notes to the trainer

- The trainer may use small groups to document a program.

 **Key steps:**

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to perform tasks described below:
You need to write comprehensive documentation for a C program that calculates the factorial of a non-negative integer. The documentation should describe the purpose of the program, how to use it, and provide details on its functionality, input/output, and any relevant notes.

Step 2: Explain the task and provide clear work instruction (Task, Time allocated)

Step 3: Ask trainees to discuss how to write the documentation of a program.

Step 4: Ask trainees to write the documentation on a written program.

Step 5: Ask trainees to present their finding to the class.

Step 6: Ask trainees to ask clarification where necessary.

Step 7: Ask trainees to read key **reading 2.7.4. Trainee Manual**

 **Points to Remember**

Documenting a program

- **Clear, concise title** to quickly identify the program's purpose.
- **Program's functionality, main purpose, and key features** to provide readers with a clear understanding of what the program achieves.

Usage Instructions

- **Compilation/Installation**: Instructions on how to compile or install the program, including any prerequisites.
- **Execution**: Steps to run the program and description of any command-line options.

Input/Output Specifications

- **Expected inputs and outputs**: Details on the format and any validation rules for inputs and outputs.

 **Application of learning 2.7.**

Rwanda meteorological Agency need a thermometer which convert temperatures from Celsius to Fahrenheit. As a c programmer Write documentation for a C program that converts temperatures from Celsius to Fahrenheit. The program should prompt the user to enter a temperature in Celsius, convert it to Fahrenheit, and display the result.

Solution:

Documentation for the Celsius to Fahrenheit Conversion Program

- **Title**

Celsius to Fahrenheit Temperature Converter

- **Description**

This C program converts a temperature from Celsius to Fahrenheit. The user inputs a temperature in Celsius, and the program computes the equivalent temperature in Fahrenheit using the formula: F=C×95+32F = C \times \frac{9}{5} + 32F=C×59+32 where FFF is the temperature in Fahrenheit and CCC is the temperature in Celsius. The program then displays the converted temperature.

- **Usage Instructions**

✓ **Compile the Program:** Use a C compiler such as gcc to compile the source code:

gcc -o temp_convertertemp_converter.c

✓ **Run the Program:** Execute the compiled program:

./temp_converter

✓ **Enter Input:**
  - When prompted, enter a temperature in Celsius.

✓ **View Output:**
  - The program will display the equivalent temperature in Fahrenheit.

- **Input/Output Specifications**

✓ **Input:**
  - A single floating-point number representing the temperature in Celsius. The input should be a valid number.

✓ **Output:**
  - The temperature in Fahrenheit, formatted to two decimal places.

- **Function Descriptions**
- **float celsiusToFahrenheit(float celsius)**
  - ✓ **Purpose:** Converts a temperature from Celsius to Fahrenheit.
  - ✓ **Parameters:**
    - float celsius - The temperature in Celsius.
  - ✓ **Returns:**
    - float - The equivalent temperature in Fahrenheit.
- **int main()**
  - ✓ **Purpose:** Entry point of the program. Manages user input, performs conversion, and displays the result.
  - ✓ **Returns:**
    - 0 - Successful execution.

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1.	Program is clearly documented	**1.1.** Title is identified		
		1.2. Description of the program is mentioned		
		1.3. Usage Instructions are written		



Learning outcome 2 end assessment

Written assessment

Multiple choice questions

1.What is the correct syntax for declaring a variable in C?

- **A.** var int x;
- **B.** int x;
- **C.** variable int x;
- **D.** int x = var;

Answer: B. int x;

2.Which of the following is the correct way to include a header file in a C program?

- **A.** #include "stdio.h"
- **B.** include <stdio.h>
- **C.** #include <stdio.h>
- **D.** #include <stdio.h.h>

Answer: C. #include <stdio.h>

3. **What will be the output of the following C code snippet?**

```
#include <stdio.h>

int main() {
   int a = 10, b = 5;
printf("%d\n", a + b);
   return 0;
}
```

A. 10
B. 15
C. 5
D. 105

Answer: B. 15

4.Which of the following data types can be used to store a floating-point number in C?

- **A.** Int
- **B.** Char
- **C.** Float
- **D.** bool

Answer: C. float

5.What is the output of the following code?

```
#include <stdio.h>

int main() {
   int x = 7;
printf("%d\n", x % 3);
   return 0;
}
```

A. 7
B. 2
C. 3
D. 1

Answer: D. 1

6.How do you declare an array of 10 integers in C?

- **A.** int arr[10];
- **B.** int arr(10);
- **C.** array int arr[10];
- **D.** int arr = [10];

Answer: A. int arr[10];

7.Which function is used to read input from the user in C?

- **A.** scanf()
- **B.** printf()
- **C.** input()
- **D.** read()

Answer: A. scanf()

8.What is the purpose of the return 0; statement in the main() function?

- **A.** To exit the program with a status code of 0
- **B.** To restart the program
- **C.** To print 0 to the console
- **D.** To create a new integer variable

Answer: A. To exit the program with a status code of 0

9.What is the correct syntax for defining a function in C?

- **A.** return_typefunction_name(parameters) { /* code */ }
- **B.** function return_typefunction_name(parameters) { /* code */ }
- **C.** function return_type { /* code */ } function_name(parameters)
- **D.** function_name(parameters) return_type { /* code */ }

Answer: A. return_typefunction_name(parameters) { /* code */ }

10. What is a function in C programming?

- **A.** A reserved keyword
- **B.** A block of code that performs a specific task
- **C.** A type of data structure
- **D.** A type of loop

Answer: B) A block of code that performs a specific task

11.Which of the following is a valid way to declare a function in C?

- **A.** int function_name[];
- **B.** function_name int();
- **C.** int function_name();
- **D.** function_name = int();

Answer: C) int function_name();

12. Which keyword is used to return a value from a function in C?

- **A.** void
- **B.** return
- **C.** break
- **D.** exit

Answer: B) return

13. Which of the following is a valid control statement in C?

- **A.** loop
- **B.** do-while
- **C.** foreach
- **D.** switch-case

Answer:B) do-while

14. What is the output of the following code?

```
int x = 5;

if (x > 3) {

printf("Hello");

} else {

printf("World");

}
```

- **A.** World
- **B.** Hello
- **C.** Error
- **D.** 5

Answer: B) Hello

15. Which control statement allows a program to execute a block of code repeatedly based on a condition?

- **A.** if
- **B.** for
- **C.** switch
- **D.** goto

Answer:B) for

16.In C, which of the following statements is used to terminate a loop?

A. continue
B. break
C. return
D. switch

Answer:B) break

17.What does the continue statement do in a loop?

A. Terminates the loop
B. Skips the current iteration and continues with the next iteration
C. Returns a value
D. Ends the program

Answer:B) Skips the current iteration and continues with the next iteration

18.Which of the following is the correct syntax for a switch statement in C?

A. switch (variable) {}
B. if (variable) {}
C. loop (variable) {}
D. case (variable) {}

Answer:A) switch (variable) {}

19.What is the purpose of the default case in a switch statement?

A. To execute when none of the other cases match
B. To stop the execution of the program
C. To execute before any other case
D. To handle all cases

Answer:A) To execute when none of the other cases match

Subjective questions

1. Explain the difference between a **for** loop and a **while** loop in C. Provide examples of when to use each.

   Answer:A for loop is typically used when the number of iterations is known beforehand. It includes initialization, a condition check, and an increment/decrement in a single line. A while loop is used when the number of

iterations is not known, and it continues executing as long as a specified condition is true.

Example:

```
// For loop example - known iterations

for (int i = 0; i< 5; i++) {

printf("%d\n", i);

}

// While loop example - unknown iterations

int n = 0;

while (n < 5) {

printf("%d\n", n);

   n++;

}
```

2. What is an array in C, and how is it declared and initialized? Provide an example.

Answer:

An array in C is a collection of variables of the same data type, stored in contiguous memory locations. It is declared by specifying the data type, the array name, and the size.

Declaration and Initialization Example:

```
// Declaration

int numbers[5];

// Initialization

int numbers[5] = {1, 2, 3, 4, 5};
```

3. How does a **switch** statement work in C, and how does it differ from an **if-else** statement? Illustrate with code examples.

   Answer:

   A switch statement allows a variable to be tested for equality against multiple values. It is generally used when a single variable needs to be compared with different constants. Unlike if-else, switch is more readable and can handle many cases more efficiently.

   Example:

```
int choice = 2;

switch (choice) {

   case 1:

printf("Choice is 1\n");

      break;

   case 2:

printf("Choice is 2\n");

      break;

   default:

printf("Invalid choice\n");

}
```

   In contrast, an if-else checks conditions sequentially:

```
if (choice == 1) {

printf("Choice is 1\n");

} else if (choice == 2) {

printf("Choice is 2\n");
```

```
} else {

printf("Invalid choice\n");

}
```

4. Describe the purpose of the **break** and **continue** statements in loops. Provide examples of how each is used.

Answer

The break statement is used to terminate the loop immediately, regardless of the condition. The continue statement skips the rest of the code inside the loop for the current iteration and moves to the next iteration.

Examples:

```
// Break example

for (int i = 0; i< 5; i++) {

   if (i == 3) {

      break;  // Loop terminates when i is 3

   }

printf("%d\n", i);

}



// Continue example

for (int i = 0; i< 5; i++) {

   if (i == 3) {

      continue;  // Skips the rest of the loop when i is 3

   }

printf("%d\n", i);
```

```
}
```

break is used when a certain condition needs to stop the loop, while continue is used to skip specific iterations without ending the loop.

5. Write a C program to find the largest number in an array of integers. Explain the logic used in your solution.

Answer:

```
#include <stdio.h>

int main() {

   int arr[] = {10, 20, 5, 30, 25};

   int n = sizeof(arr) / sizeof(arr[0]);

   int max = arr[0];



   for (int i = 1; i< n; i++) {

      if (arr[i] > max) {

         max = arr[i];

      }

   }



printf("The largest number is %d\n", max);

   return 0;

}
```

6. What are nested loops, and when are they used in C programming? Provide an example of a situation where nested loops would be necessary.

Answer:

Nested loops are loops inside other loops. They are used when performing operations on multi-dimensional data structures or when a repetitive task needs to occur within another repetitive task.

Example: Printing a 5x5 matrix.

```
for (int i = 0; i< 5; i++) {

   for (int j = 0; j < 5; j++) {

printf("* ");

   }

printf("\n");

}
```

Here, the outer loop controls the rows, and the inner loop controls the columns, printing a 5x5 grid of *.

7. Discuss the concept of an infinite loop. How can an infinite loop be created in C, and how can it be terminated?

Answer:

An infinite loop is a loop that continues indefinitely because the condition never becomes false. It can be created using a while or for loop without a termination condition.

Example:

```
while (1) {

   // Code here runs forever

}
```

An infinite loop can be terminated by using a break statement or by an external action (like user input or system signals).

8. Explain how multidimensional arrays are used in C. Provide an example of how a 2D array can be used to store and access data.

Answer:

Multidimensional arrays are arrays with more than one dimension. A common example is a 2D array, which is used to represent matrices or tables.

Example:

```
int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};

// Accessing elements

printf("%d\n", matrix[0][2]); // Output: 3

printf("%d\n", matrix[1][0]); // Output: 4
```

Here, matrix is a 2x3 array storing integers. Elements are accessed using two indices: the row and column.

9. What are the limitations of using arrays in C? How do arrays differ from other data structures like linked lists?

Answer:

Limitations of Arrays:

- Fixed Size: Once declared, the size of an array cannot be changed.
- Memory Allocation: Memory is allocated in contiguous blocks, which can lead to wasted memory or fragmentation.
- Insertion/Deletion: Inserting or deleting elements is costly as it requires shifting elements.

Differences from Linked Lists:

- Arrays provide fast access using index, while linked lists allow dynamic resizing and efficient insertion/deletion.
- Arrays use contiguous memory; linked lists use scattered memory locations with pointers connecting elements.

Practical assessment

1. Write a C program to find the sum of all integers from 1 to n, where n is provided by the user.

Answer:

```
#include <stdio.h>
int main() {
   int n, sum = 0;
   // Prompt user for input
printf("Enter a positive integer: ");
scanf("%d", &n);
   // Calculate the sum
   for (int i = 1; i<= n; i++) {
      sum += i;
   }
   // Display the result
printf("The sum of all integers from 1 to %d is %d\n", n, sum);
   return 0;
}
```

Checklist

SN	CRITERIA	INDICATORS	YES	NO
	The string is well reversed.	**1.1.** The integer is declared.		
		1.2. Prompt user to input a integer		
		1.3. The loop to calculate the sum is well mentioned		
		1.4. The sum is displayed		

2. Write a C program to check if a given number is a prime number.

Answer:

```
#include <stdio.h>
#include <stdbool.h>
```

```
bool isPrime(int num) {
   if (num <= 1) return false;
   for (int i = 2; i * i<= num; i++) {
      if (num % i == 0) return false;
   }
   return true;
}
int main() {
   int num;
   // Prompt user for input
printf("Enter a positive integer: ");
scanf("%d", &num);
   // Check if the number is prime
   if (isPrime(num)) {
printf("%d is a prime number.\n", num);
   } else {
printf("%d is not a prime number.\n", num);
   }
   return 0;
}
```

Checklist/solution for application

SN	CRITERIA	INDICATORS	YES	NO
	The prime number is well displayed.	**1.1.** The integer is declared.		
		1.2. Prompt user to input a number		
		1.3. The condition structure to check if the given number is prime is declared		
		1.4. The prime number is displayed		

3. Write a C program to reverse a string entered by the user.

Answer:

```
#include <stdio.h>
#include <string.h>
int main() {
   char str[100], reversed[100];
   int length, i, j;
   // Prompt user for input
printf("Enter a string: ");
fgets(str, sizeof(str), stdin);
   str[strcspn(str, "\n")] = '\0'; // Remove newline character if present
   length = strlen(str);
   j = 0;
   // Reverse the string
   for (i = length - 1; i>= 0; i--) {
      reversed[j++] = str[i];
   }
   reversed[j] = '\0';
   // Display the reversed string
printf("Reversed string: %s\n", reversed);
   return 0;
}
```

Checklist/solution for application

SN	CRITERIA	INDICATORS	YES	NO
1.	**The string is well reversed.**	**1.1.** The character is declared.		
		1.2. Prompt user to input a string		
		1.3. The loop to reverse a string is mentioned		
		1.4. The reversed string is displayed		

4. Write a C program to calculate the factorial of a non-negative integer using recursion.

Answer:

```
#include <stdio.h>


int factorial(int n) {
   if (n == 0) return 1;
   return n * factorial(n - 1);
}
int main() {
   int num;
   // Prompt user for input
printf("Enter a non-negative integer: ");
scanf("%d", &num);
   if (num < 0) {
printf("Factorial is not defined for negative numbers.\n");
   } else {
printf("Factorial of %d is %d\n", num, factorial(num));
   }
   return 0;
}
```

Checklist/solution for application

SN	CRITERIA	INDICATORS	YES	NO
	The factorial of a number is well calculated.	**1.1.** The formulae for calculating the factorial of a number is mentioned.		
		1.2. Prompt user to input a number		
		1.3. The condition for checking if the number is negative or not is mentioned		
		1.4. The results is displayed		

5. Write a C program to find the largest and smallest number in an array of integers.

Answer:

```
#include <stdio.h>
int main() {
   int n;
   // Prompt user for number of elements
printf("Enter the number of elements: ");
scanf("%d", &n);
   int arr[n];
   int i, largest, smallest;
   // Prompt user to enter the array elements
printf("Enter %d integers:\n", n);
   for (i = 0; i< n; i++) {
scanf("%d", &arr[i]);
   }
   largest = smallest = arr[0];
   // Find the largest and smallest numbers
   for (i = 1; i< n; i++) {
      if (arr[i] > largest) {
         largest = arr[i];
      }
      if (arr[i] < smallest) {
         smallest = arr[i];
      }
   }
  // Display the results
printf("Largest number: %d\n", largest);
printf("Smallest number: %d\n", smallest);
  return 0;}
```

Checklist/solution for application

SN	CRITERIA	INDICATORS	YES	NO
1.	The array is correctly sorted	**1.1.** Prompt user for number of elements is displayed		
		1.2. Prompt user to enter the array elements is displayed		
		1.3. The loop to find the greatest number is mentioned		
		1.4. The results is displayed		

6. Write a C program to sort an array of integers in ascending order using the bubble sort algorithm.

Answer:

```
#include <stdio.h>
void bubbleSort(int arr[], int n) {
   for (int i = 0; i< n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (arr[j] >arr[j + 1]) {
            // Swap arr[j] and arr[j + 1]
            int temp = arr[j];
arr[j] = arr[j + 1];
arr[j + 1] = temp;
         }
      }
   }
}
int main() {
   int n;
   // Prompt user for number of elements
printf("Enter the number of elements: ");
scanf("%d", &n);
   int arr[n];
   // Prompt user to enter the array elements
```

```
printf("Enter %d integers:\n", n);
   for (int i = 0; i< n; i++) {
scanf("%d", &arr[i]);
   }

   // Sort the array
bubbleSort(arr, n);
   // Display the sorted array
printf("Sorted array in ascending order:\n");
   for (int i = 0; i< n; i++) {
printf("%d ", arr[i]);
   }
printf("\n");
   return 0;
}
```

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1.	The array is well sorted	**1.1.** Prompt user for number of elements is displayed		
		1.2. Prompt user to enter the array elements is displayed		
		1.3. The array is sorted in ascending order is displayed.		

END



Further information to the trainer

A. Books

Graham, D. L. (2019). C Programming Language: A Step by Step Beginner's Guide to Learn C Programming. Amazon.

King, K. (2008). C Programming: A Modern Approach 2nd Edition. Amazon.

McGrath, M. (2018). C Programming in easy steps, 5th Edition. Amazon.

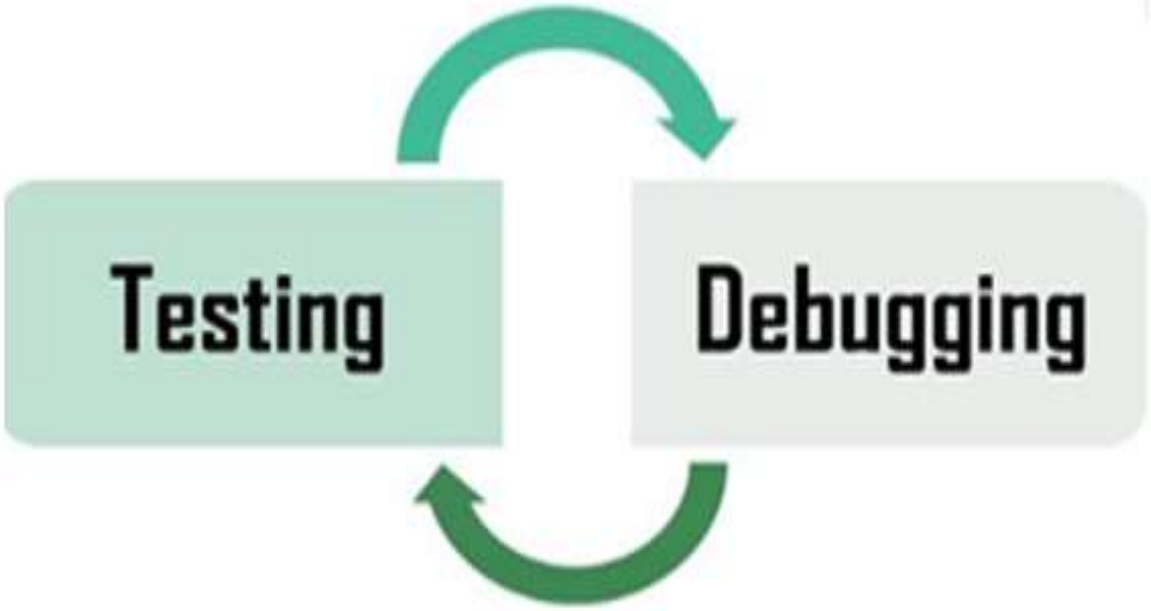
Miller, G. P. (2013). C Programming Absolute Beginner's Guide. Amazon.

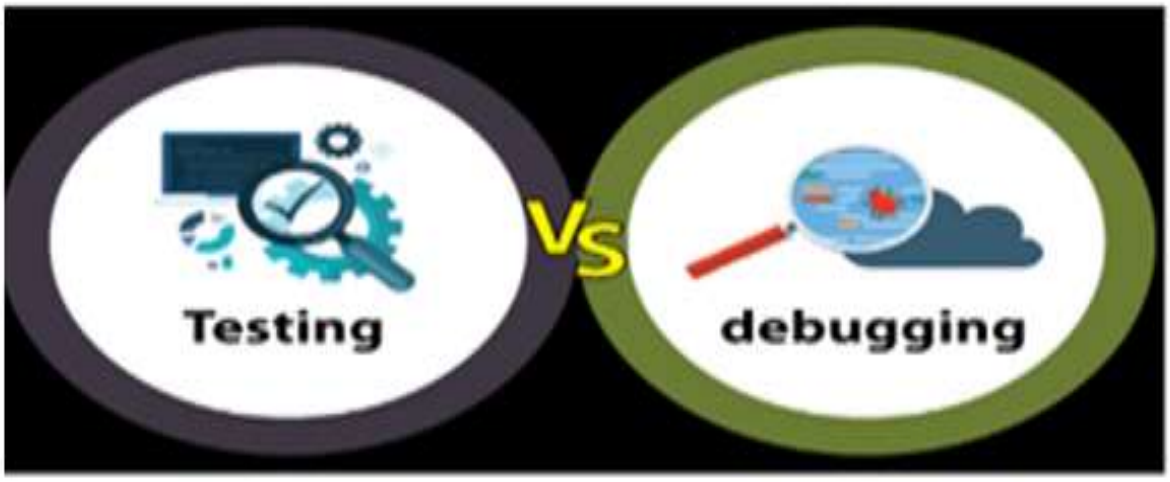
Norton, W. (2021). The C Programming Language (Simplified for Beginners). Amazon.

Thareja, R. (2011). Data Structures Using C. Amazon.

B. Websites

https://engineerstutor.com/2020/10/05/solved-assignment-problems-in-c-with-algorithm-and-flowchart/.
https://www.zenflowchart.com/guides/flowchart-in-c-programming.
https://courses.minia.edu.eg/Attach/16036flowchart-algorithm-manual.pdf
https://ccsuniversity.ac.in/bridge-library/pdf/btech-cs/Functions%2025,26,27,28,-converted.pdf
https://www.tutorialspoint.com/cprogramming/pdf/c_functions.pdf
https://www2.seas.gwu.edu/~bell/csci1121/lectures/functions.pdf
https://intranet.cb.amrita.edu/sites/default/files/1.6%20Function.pdf
https://www.albany.edu/faculty/dsaha/teach/2017Spring_CEN360/slides/lec08.pdf
https://www.simplilearn.com/tutorials/c-tutorial/array-in-c#:~:text=Array%20in%20C%20can%20be,data%20types%20too%20like%20structures.
https://www.tutorialspoint.com/cprogramming/c_arrays.htm
https://www.geeksforgeeks.org/c-arrays/
https://www.digitalocean.com/community/tutorials/initialize-an-array-in-c


Testing
Debugging


Testing
Vs
debugging

Indicative contents
3.1. Identification of errors **3.2. Compilation of the C program** **3.3. Test of the C program**

Key Competencies for Learning Outcome 3: Perform Program Debugging

Knowledge	Skills	Attitudes
• Description of errors types. • Description of types of compilers • Description of debugging techniques and tools you can use when working with C programming • Description of the types of testing	• Selecting types of errors, including syntax errors, logical errors, and runtime errors in a c program • Applying debugging tools like GDB or integrated development environment (IDE) debuggers. • • Debugging a c program • Selecting of different types of compilers in c program • Testing a c program	• Having an innovative • Having Creativity • Having Critical thinking • Having Teamwork • Being Problem Solver • Being Patient • Having Punctuality • Having Curiosity

Duration: 10hrs

Learning outcome 3 objectives:

By the end of the learning outcome, the trainees will be able to:

1. Describe correctly different program errors in computer programming

2. Compile C program in accordance with the program instructions.

3. Debug correctly the errors in a c program.

4. check errors program correctly based on programming instructions.

5. Test correctly based on programming instructions.

Resources

Equipment	Tools	Materials
● Computers ● Projector ● Projection screen	● Internet ● C programming IDE	● Pens ● Books ● Paper

Advance Preparation:

Before delivering this learning outcome, you are recommended to:

- Prepare sample drawings (Illustration, images or Videos) to be used as didactic materials.
- Avail testing tools
- Organize classroom and Workshop and/or electrical laboratory.

Indicative content 3.1: Identification of Errors

Duration: 5 hrs

Theoretical Activity 3.1.1: Describe types of errors

Notes to the trainer:

- Trainer may small groups may be used to describe the types of errors.
- Avail examples of written algorithms to describe types of arrays.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and request learners to respond to the following questions:
  - **i.** What is an error in c programming?
  - **ii.** How to read an error in c programming?
  - **iii.** What are the different types of errors?

Step 2: Monitor the discussion and asks trainees to write the findings on paper

Step 3: Ask trainees to present their findings to the whole class.

Step 4: Provide an expert view and relate it to what they presented.

Step 5: Address any questions or concerns.

Step 6: Orient them to read the key reading **3.1.1**

Points to Remember

Types of Errors in C Programming

In C programming, an error refers to any deviation from the intended or expected behavior of a program. Errors can occur at various stages of development and execution and are categorized into three main types: syntax errors, logical errors, and runtime errors.

- **Syntax Errors:**
  - ✓ Violations of the rules of the C programming language. They occur when the code does not conform to the syntax of C.
- **Logical Errors:**

- ✓ Flaws in the logic or design of a program, leading to undesired outcomes or

- **Runtime Errors:**
  - ✓ Occur during the execution of a program due to unexpected conditions.



Practical Activity 3.1.2: Correcting errors



Notes to the trainer

- The trainer may use individual based;
- you are supposed to correct errors.



Key steps:

While delivering this activity, pass through the following steps:

Step 1:	Introduce the topic and ask learners to perform tasks described below: In your computer lab, suppose that you're working on a simple C program that reads a series of numbers from the user, calculates their sum, and then displays the result. However, there are several errors in the code, and your task is to identify practical techniques for correcting errors and correct them.
Step 2:	Explain the task and provide clear work instruction (Task, Time allocated)
Step 3:	Demonstrate how to do the given tasks by identifying the founded errors.
Step 4:	Demonstrating, how to correct errors.
Step 5:	Compile for checking if errors are corrected according to the finding answers.
Step 6:	Ask trainees to read **keyreading 3.1.2**



Points to Remember

Correcting Errors in C Programming

- Understanding Error Messages
- Inspect the Code
- Use a Debugger
- Print Statements
- Isolate the Problem

- Divide and Conquer



Application of learning 3.1.

You are an engineer working on an embedded system project for a weather monitoring station. The station collects temperature readings from multiple sensors installed in different locations (e.g., on different floors of a building or at various points in a large area). Your task is identify the errors in the segment of code.

```
#include <stdio.h>

        int main() {

           int arr[5] = {1, 2, 3, 4, 5};

           for (int i = 0; i<= 5; i++) {

        printf("%d ", arr[i]);

           }

           return 0;

        }
```

Task:

1. Identify the error in the code and explain why it occurs.
2. How would you correct the error to prevent the program from accessing out-of-bounds memory?

Answer:

1. Identify the Error and Explain Why It Occurs

Error:
The error in the code is found in the for-loop condition:

```
for (int i = 0; i<= 5; i++) {
```

The loop iterates while i is less than or equal to 5. Since array indices in C are zero-based, valid indices for arr range from 0 to 4 (5 elements in total). However, the condition i<= 5 allows i to take the value 5, which is an out-of-bounds index for arr.

When i becomes 5, the code attempts to access arr[5], which does not exist in the array. Accessing an out-of-bounds element in an array leads to undefined behavior, which can cause the program to crash, produce incorrect results, or access unintended memory.

2. Correcting the Error

Correction:
To prevent the program from accessing out-of-bounds memory, you should change the loop condition so that i only ranges from 0 to 4, which are the valid indices for the array arr. The corrected loop should be:

```
for (int i = 0; i< 5; i++) {
```

Corrected Code:

```
#include <stdio.h>

int main() {

   int arr[5] = {1, 2, 3, 4, 5};

   for (int i = 0; i< 5; i++) {

printf("%d ", arr[i]);

   }

   return 0;

}
```

Checklist/solution for application

SN	CRITERIA	INDICATORS	YES	NO
1.	Errors are well corrected	**1.1.** The error message is understandable		
		1.2. The problem is isolated		
		1.3. Divide and conquer is used		
		1.4. The code is inspected		
		1.5. Expected Result is displayed		

Indicative content 3.2: Compilation of the C Program

Duration: 3 hrs

Theoretical Activity 3.2.1: Describe types of compilers

Notes to the trainer:

- Trainer small groups may be used to describe the types of compilers.

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the activity and request learners to respond to the following questions:

i. What is a compiler?
ii. What does the compiler do exactly?
iii. identify the different types of compilers

Step 2: Monitor the discussion and asks trainees to write the findings on paper

Step 3: Ask trainees to present their findings to the whole class.

Step 4: Provide an expert view and relate it to what they presented.

Step 5: Address any questions or concerns.

Step 6: Orient them to read the key reading

Points to Remember

What is a Compiler?

A **compiler** is software that converts high-level programming language code (like C) into machine-level language. It checks the code for errors and reports them if the code does not follow language rules.

Types of Compilers:

- **Borland Turbo C**: A basic, fast C compiler known for its small size, re-released as freeware in 2006.
- **Tiny C Compiler (TCC)**: Optimized for slow computers with limited disk space, known for its small size and fast compilation speed.

- **Portable C Compiler (PCC)**: An influential compiler from the mid-1970s, designed for portability and error checking with machine-dependent source files.
- **GCC (GNU Compiler Collection)**: Supports multiple languages, initially released for C, and known for extensive optimizations like dead code elimination.



Practical Activity 3.2.2: Compilation techniques



Notes to the trainer

- The trainer may use small group
- Avail compilation tools



Key steps:

While delivering this activity, pass through the following steps:

Step 1:	Introduce the topic and ask learners to perform tasks described below: Compile and run a simple C program that prints "Hello, World!" to the console.
Step 2:	Explain the task and provide clear work instruction (Task, Time allocated)
Step 3:	Ask trainees to write the source code.
Step 4:	Ask trainees to open terminal/command prompt
Step 5:	Ask trainees to Compile the Program
Step 6:	Repeat the whole process to be clear and concise
Step 7:	Ask trainees to read **keyreading 3.2.2**



Points to Remember

Compilation techniques

- **Preprocessing**: Processes preprocessor directives and macros, performing text substitution to prepare the code for further processing.
- **Lexical Analysis (Scanning)**: Breaks down the source code into tokens, which are the smallest units of meaning in the programming language.
- **Syntax Analysis (Parsing)**: Analyzes the sequence of tokens to ensure they follow the syntax rules of the language, constructing a parse tree or abstract syntax tree (AST).

- **Semantic Analysis**: Verifies the meaning and correctness of the code, performing type checking and enforcing semantic rules.



Application of learning 3.2.

A parser is responsible for checking the syntactic structure of the code according to the rules of the programming language. Consider the following code snippet:

```
if (x > 0 {
printf("Positive");
}
```

Task:

1. Identify the syntax error in the code and explain why it would be flagged by the parser.
2. How does a parser use a parse tree or abstract syntax tree (AST) to represent the structure of this code?

Answer:

The syntax error in the provided code snippet is in the if statement:

```
if (x > 0 {
printf("Positive");
}
```

Error: There is a missing closing parenthesis) after x > 0.

Parse Tree or Abstract Syntax Tree (AST) Representation

A parser uses a **parse tree** or an **abstract syntax tree (AST)** to represent the syntactic structure of the code.

- **Parse Tree:** A parse tree represents the hierarchical structure of the code based on the grammar of the programming language. Each node in the parse tree corresponds to a grammatical construct (e.g., expressions, statements), and the tree is built according to the rules of the language. The parse tree is usually more detailed, reflecting the precise syntactic structure, including all symbols like parentheses.
- **Abstract Syntax Tree (AST):** An AST is a simplified version of the parse tree, where certain syntactic details are omitted to focus on the essential structure of the code.

It abstracts away some of the grammatical rules, leaving only the essential components like operators, operands, and control structures.

For the given code (after fixing the syntax):

```
if (x > 0) {
printf("Positive");
}
```

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1.	Test case is correctly identified	**1.1.** Abstract Syntax Tree is identified		
		1.2. Error is correctly mentioned		

Indicative content 3.3: Test of the C Program

Duration: 2 hrs

Theoretical Activity 3.3.1: Design of effective test cases

Notes to the trainer:

- Trainer may use group discussion
- Avail test case examples

Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to perform tasks described below:

Write a c program to display a simple calculator and write all possible test cases of this program.

Step 2: Ask trainees to discuss on the given tasks in their groups.

Step 3: Ask trainees to write down the finding from the discussion.

Step 4: Ask trainees to present the findings.

Step 5: Ask clarification if any.

Step 6: Ask trainees to read the key reading **3.3.1**

Points to Remember

- **Test Case ID:** Assign a unique identifier.
- **Description:** Briefly describe what the test case is verifying.
- **Preconditions:** State any conditions that must be met before executing the test case.
- **Test Steps:** Outline the sequence of actions to be performed.
- **Expected Result:** Define the anticipated outcome.

- **Postconditions:** Describe the state after the test has been executed.



Practical Activity 3.3.2: Debugging techniques



Notes to the trainer

- The trainer may use small groups



Key steps:

While delivering this activity, pass through the following steps:

Step 1: Introduce the topic and ask learners to perform tasks described below:

Write a c program to find the factorial of a number, you are requested to identify the debugging techniques you have used.

Step 2: Ask trainees to Write the source code of a given scenario in their groups
Step 3: Ask trainees to compile the written source code
Step 4: Ask trainees to debug the founded bugs if any
Step 5: Read the key reading **3.3.2**



Points to Remember

Debugging Techniques:

- **Print Debugging**: Use print statements to display variable values and program flow.
- **Debugger Tools**: Utilize tools like GDB or IDE features to set breakpoints, step through code, and inspect variables.
- **Breakpoints**: Pause execution at specific points to inspect variables and program state.
- **Watch and Evaluate Expressions**: Monitor and evaluate variable values or expressions during execution.



Application of learning 3.3.

You have opened a savings account at a local bank, and you want to know how much interest you will earn over a certain period. The bank offers a simple interest rate of 5% per year. You decide to write a C program to calculate the interest you will earn based on the amount of money (principal) you deposit, the interest rate, and the time period in years.For this task, please complete the following:

a. Identify possible test cases.

b. Describe the debugging techniques employed.

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1.	Test case is correctly identified	**1.1.** Test Case ID is correctly identified		
		1.2. Description is correctly described		
		1.3. Preconditions are correctly written		
		1.4. Test Steps are correctly mentioned		
		1.5. Expected Result is displayed		
2.	Debugging techniques are correctly mentioned	**2.1.**Print Debugging is well used in program flow		
		2.2. Debugger Tools are well used		
		2.3. Static Analysis Tools are well used		
		2.5 Code Review is well described		



Learning outcome 3 end assessment

Written assessment

Multiple choice questions

Types of Errors in Programming

1. **Which of the following is an example of a runtime error in C programming?**

    A) Using an undeclared variable

    B) Dividing a number by zero

    C) Missing a semicolon at the end of a statement

    D) Incorrect use of function syntax

Answer: B) Dividing a number by zero

2. **What type of error is caused by violations of the rules of the programming language syntax?**

    A) Runtime error

    B) Logical error

    C) Syntax error

    D) Semantic error

Answer: C) Syntax error

3. **Which type of error is the most difficult to detect and fix because the program still runs but gives incorrect output?**

A) Compilation error

    B) Logical error

    C) Syntax error

    D) Semantic error

Answer: B) Logical error

4. **What type of error will the following code produce? printf("Hello World)**

    A) Syntax error

    B) Runtime error

C) Logical error

D) None of the above

Answer: A) Syntax error

2. Compilation Techniques

5. **Which of the following is the first step in the C compilation process?**

    A) Linking

    B) Preprocessing

    C) Code Optimization

    D) Assembling

Answer: B) Preprocessing

6. **What is the purpose of the linker in the compilation process?**

    A) To check syntax errors

    B) To combine multiple object files into a single executable

    C) To translate code into machine language

    D) To optimize the code for performance

Answer: B) To combine multiple object files into a single executable

7. **Which compilation technique translates high-level code into machine code one line at a time, while the program is running?**

    A) Interpreter

    B) Compiler

    C) Assembler

    D) Linker

Answer: A) Interpreter

8. **Which phase of compilation involves translating assembly code to machine code?**

    A) Linking

    B) Preprocessing

    C) Assembling

    D) Optimization

Answer: C) Assembling

3. Debugging Tools

9. **Which of the following tools is commonly used for debugging C programs on Unix-like systems?**

    A) Valgrind

    B) Git

    C) NetBeans

    D) Makefile

Answer: A) Valgrind

10. **What does a debugger tool primarily help with?**

    A. Writing code faster
    B. Detecting syntax errors
    C. Testing program performance
    D. Finding and fixing runtime errors

Answer: D) Finding and fixing runtime errors

11. **Which debugging tool allows you to step through your program line-by-line to check its execution flow?**

    A. Compiler
    B. Debugger
    C. Profiler
    D. Assembler

Answer: B) Debugger

12. **Which of the following is NOT a feature of most modern debugging tools?**

    A) Breakpoints

    B) Memory leak detection

    C) Automated code generation

    D) Stack trace analysis

Answer: C) Automated code generation

13. **Which tool is used to analyze memory management problems, like memory leaks and invalid memory access in C programs?**

    A) GDB (GNU Debugger)

B) GCC (GNU Compiler Collection)

C) Valgrind

D) Make

Answer: C) Valgrind

14. **What is the main use of setting breakpoints while debugging a program?**

A) To make the code run faster

B) To stop the execution of the program at specific points

C) To compile the program

D) To link multiple object files

Answer: B) To stop the execution of the program at specific points

Subjective questions

1. What is the purpose of a compiler in the context of C programming?

Answer: A compiler in C programming translates source code written in the high-level C language into machine code or lower-level code that the computer's processor can execute. It checks the code for syntax and semantic errors, performs optimizations, and generates an executable file.

2. Describe the main phases of the compilation process in C programming.

Answer: The main phases of the C compilation process are:

1. **Preprocessing:** Handles preprocessor directives (e.g., #include, #define) and prepares the code for compilation by performing text substitution and macro expansion.
2. **Lexical Analysis:** Breaks down the source code into tokens, which are the smallest units of meaning (keywords, identifiers, operators, etc.).
3. **Syntax Analysis:**Analyzes the tokens to ensure they follow the syntax rules of the language and constructs a parse tree or abstract syntax tree (AST) representing the code's structure.
4. **Semantic Analysis:** Ensures that the code makes logical sense, checking for type consistency, valid variable declarations, and correct function calls.
5. **Intermediate Code Generation:** Produces an intermediate representation of the code, which is platform-independent and easier to optimize.
6. **Optimization:** Applies various techniques to improve the efficiency of the generated code, such as eliminating dead code or optimizing loops.

7. **Code Generation:** Translates the intermediate code into machine code or assembly language specific to the target architecture.
8. **Linking:** Combines multiple object files and libraries into a single executable, resolving references between different modules and external libraries.

3. What is the role of a linker in the C compilation process?

Answer: The linker combines multiple object files and libraries into a single executable file. It resolves references between different parts of the program, such as function calls and variable accesses, and ensures that all code and data are properly linked together. The linker also handles the inclusion of external libraries and addresses any inter-module dependencies.

4. Explain the difference between syntax errors and semantic errors in C programming.

Answer:

- **Syntax Errors:** These occur when the code violates the grammatical rules of the C language. Examples include missing semicolons, unmatched parentheses, or incorrect keyword usage. Syntax errors are usually detected during the compilation phase and prevent the code from being compiled.
- **Semantic Errors:** These occur when the code is syntactically correct but semantically incorrect, meaning it does not make logical sense. Examples include type mismatches, invalid function calls, or incorrect use of variables. Semantic errors are usually detected during the semantic analysis phase of compilation.

5. What is the purpose of the #include directive in a C program?

Answer: The #include directive is used to include the contents of a specified file into the source code file before compilation. It is commonly used to include standard library headers or user-defined header files. This directive allows the program to use functions, macros, and definitions declared in the included files.

6. How does optimization improve the performance of a compiled C program?

Answer: Optimization improves the performance of a compiled C program by applying various techniques to make the code more efficient. These techniques can include:

- **Dead Code Elimination:** Removing code that does not affect the program's outcome.
- **Loop Unrolling:** Reducing the number of iterations by increasing the number of operations within a loop.
- **Constant Folding:** Evaluating constant expressions at compile time rather than runtime.
- **Inlining:** Replacing function calls with the actual function code to reduce the overhead of function calls.

These optimizations can lead to faster execution, reduced memory usage, and overall improved performance of the program.

7. What is the significance of the main function in a C program?

Answer: The main function is the entry point of a C program. It is where the execution of the program begins. Every C program must have a main function, and it typically returns an integer value indicating the success or failure of the program's execution. The main function can also take command-line arguments, which can be used to provide input to the program at runtime.

8. What is a preprocessor directive, and provide an example?

Answer: A preprocessor directive is a command that is processed by the C preprocessor before the actual compilation of the code begins. Preprocessor directives are used to include files, define macros, and conditionally compile code. An example of a preprocessor directive is #define, which is used to create macros:

#define PI 3.14159

This directive defines a constant PI with the value 3.14159, which can be used throughout the code wherever PI is referenced.

Practical assessment

1. **Write a C program, which calculates the area of a rectangle.**

Your tasks are:

1. Compile the C program using a C compiler.
2. Identify and explain any errors or warnings that occur during the compilation process.
3. Describe the steps taken during the compilation and the purpose of each step.

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1.	Compiler a c program is well done	**1.1.** Compilation is done		
2.	Error is well identified	**2.1** Error warning is identified		
		2.2. The steps for identifying error during the compilation		

C Program:

```
#include<stdio.h>

intmain() {
int length, width;
printf("Enter the length of the rectangle: ");
scanf("%d", &length);
printf("Enter the width of the rectangle: ");
scanf("%d", &width);

int area = length * width;
printf("The area of the rectangle is: %d\n", area);

return0;
}
```

Answer:

1. Compile the C Program:

To compile the given C program, use a C compiler such as gcc. Open a terminal and navigate to the directory where the C program file (e.g., rectangle.c) is located. Run the following command:

```
gcc -o rectangle rectangle.c
```

Here, gcc is the compiler command, -o rectangle specifies the name of the output executable file, and rectangle.c is the source code file.

2. Identify and Explain Errors or Warnings:

In this case, the provided program is straightforward and should compile without errors. However, let's discuss possible issues that might occur in a similar scenario:

- **Syntax Errors:** If there were any syntax errors (e.g., missing semicolons or incorrect syntax), the compiler would display an error message indicating the line number and the nature of the error. For example, a missing semicolon after the printf statement would generate an error like:

  ```
  rectangle.c: In function ‘main’:
  rectangle.c:11:5: error: expected ‘;’ before ‘return’
  ```

return0;

- **Warnings:** The compiler might produce warnings if there are potential issues that do not stop compilation but could lead to runtime problems. For instance, using scanf with %d requires valid integer input. A warning might occur if improper format specifiers are used.

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1.	Compiler a c program is well done	**1.2.** Compilation is done		
2.	Error is well identified	**2.1.**Error warning is identified		
		2.3. The steps for identifying error during the compilation		

3. Describe the Steps Taken During Compilation

- **Preprocessing:** The #include <stdio.h> directive is handled by the preprocessor, which includes the standard input-output library header file into the source code. This step processes any preprocessor directives and prepares the code for the next phase.
- **Lexical Analysis:** The compiler breaks down the source code into tokens, such as keywords (int, return), identifiers (length, width), operators (*, =), and literals ("Enter the length of the rectangle: ").
- **Syntax Analysis:** The compiler checks the structure of the code to ensure that it adheres to the C language syntax rules. It constructs a parse tree or abstract syntax tree (AST) to represent the program's structure.
- **Semantic Analysis:** The compiler verifies that the code is semantically correct. For example, it ensures that the variable length and width are properly declared and used, and that the calculation of the area is valid.
- **Intermediate Code Generation:** The compiler may generate intermediate code that is easier to optimize and translate into machine code.
- **Optimization:** The compiler applies optimization techniques to improve the efficiency of the generated code.
- **Code Generation:** The compiler translates the intermediate code into machine code or assembly language, specific to the target architecture.

- **Linking:** If necessary, the compiler links the object code with libraries to create the final executable. In this simple example, there are no additional libraries to link, so the executable is created directly.
- **Executable Output:** The gcc command generates an executable file named rectangle. You can run this file to execute the program.

Checklist

SN	CRITERIA	INDICATORS	YES	NO
1. Compiler a c program is well done		**1.3.** Preprocessing is well done		
		1.4. Executable Output is well displayed		
		1.5. Semantic Analysis is well completed		
		1.6. Syntax Analysis is well completed		

END



Further information to the trainer

A. Books

Graham, D. L. (2019). C Programming Language: A Step by Step Beginner's Guide to Learn C Programming. Amazon.

King, K. (2008). C Programming: A Modern Approach 2nd Edition. Amazon.

McGrath, M. (2018). C Programming in easy steps, 5th Edition. Amazon.

Miller, G. P. (2013). C Programming Absolute Beginner's Guide. Amazon.

Norton, W. (2021). The C Programming Language (Simplified for Beginners). Amazon.

Thareja, R. (2011). Data Structures Using C. Amazon.

B. Websites

https://www.bing.com/ck/a?!&&p=be8bdb80a0c6fbf0JmltdHM9MTcwMDM1MjAwMCZpZ3VpZD0wMTU4N2M4NC03NWVlLTZlOWItMzIwYS02ZjQ0NzQ2YzZmM2EmaW5zaWQ9NTE5OA&ptn=3&ver=2&hsh=3&fclid=01587c84-75ee-6e9b-320a-6f44746c6f3a&psq=scenario+application+of+learning+of+debugging+c+programming&u=a1aHR0cHM6Ly9jcy5icm93bi5lZHUvY291cnNlcy9jc2NpMTMxMC8yMDIwL2Fzc2lnbi9sYWJzL2xhYjEuaHRtbA&ntb=1

https://www.google.com/search?client=firefox-b-d&q=compiler+in+c+programming

https://www.educba.com/best-c-compilers/

https://www.quora.com/What-are-the-different-types-of-compilers-in-C

https://www.google.com/search?client=firefox-b-d&q=common+debugging+techniques+and+tools+you+can+use+when+working+with+C+programming

RTB | RWANDA TVET BOARD

October,2024